CHAPTER 1

INTRODUCTION

INTRODUCTION-

Cancer is a wide term. It labels the illness that outcome once cellular changes cause the uncontrolled growth and division of cells. Most of the body's cells have particular functions and fixed lifetimes. However, cell death is part of a natural phenomenon called apoptosis. A cell takes directions to die so that the body can substitute it with a newer one that functions better. Cancerous cells lack the mechanisms that train them to stop dividing and to die. Thus, they grow in the body, using oxygen and nutrients that would usually feed other cells. Cancerous cells can form tumors, damage the immune system and cause other deviations that prevent the body from functioning right. Lung cancer is a malignant lung tumor considered by uncontrolled cell growth in lung tissues. Lung cancer is the primary cause of cancer-related death.[1]

According to the survey of World Health Organization (WHO), Lung cancer was the second most leading cause of death in 2015 and it is on fifth rank in 2017. It is most common in smokers accounting 85% of cases among all. So many Computer Aided Diagnosis (CAD) Systems are developed in recent years. Detection of lung cancer at early stage is necessary to prevent deaths and to increase survival rate. Lung nodules are the small masses of tissues which can be cancerous or noncancerrous also called as malignant or benign. Benign tissues are most commonly non-cancerous and does not have much growth where malignant tissues grows very fast and can affect to the other body parts and are dangerous to health.

For medical imaging so many different types of images are used but Computer Tomography (CT) scans are generally prefered beacause of less noise. Deep learning is proven to be the best method for medical imaging, feature extraction and classification of objects. Several types of deep learning architectures are introduced by so many researchers to classify the lung cancer. In this study, 3D multipath VGG-like network is proposed with classifications. One classification is of lung nodules and non-nodules and other is of benign nodules and malignant nodules. This study also adapts U-Net architecture for segmentation of lung CT

The primary goal of our research is to diagnose the presence of lung cancer cells based on attributes, which are set of human symptoms, and information. The study explores the possibility of using an Artificial Neural Network model to detect the presence of a lung cancer in someone’s body. The purposes of this study are:

- To recognize some appropriate factors that cause lung cancer

- To model an Artificial Neural Network that can be used to detect the presence of lung cancer

Artificial neural networks (ANNs) are alike to our neural networks and offer a quite good

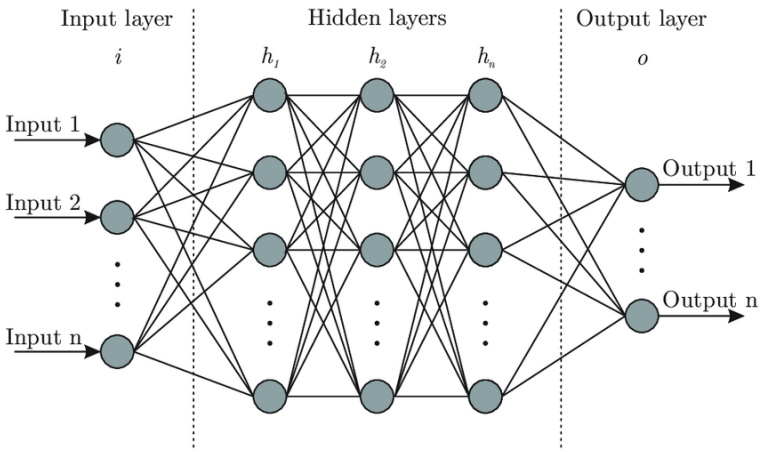


Fig 1.1 Artificial Neural Network

technique, which solves the problem of classification and prediction[2]. An ANN is a mathematical model that is encouraged by the organization and functional feature of natural neural networks[3], Neural networks involve input and output layers, as well as (in most cases) hidden layers that transform the input into something so the output layer can use[4]. When a neural network used for cancer detections, the ANN Model go through two levels, training and validation. First, the network is trained on a dataset. Then the weights of the connections between neurons are fixed so the network is validated to determine the classifications of a new dataset[5].

1.1 Prior Work

So many different types of deep learning architectures are proposed in recent researches for image segmentation and object classification in images. Some architectures are also proposed for medical imaging disease diagnosis. Two architectures among them are adapted and modified in this study.

Convolutional Neural Network is also known as CNN and ConvNet. CNN is useful for feature extraction and classification of objects in the image. This CNN is nothing but a stack of different layers. Convolutional layer and pooling layer has the responsibility of feature extraction of objects provided to CNN where convolutional layer extract the

features and pooling layer selects the important features from them also known as subsampling of convolved features. There are two types of pooling as max pooling and average pooling, but max pooling is widely used in so many researches where maximum value among the values in pooling window is selected as sampled feature from convolved features.

Activation function is used to determine output of neural network. It squash input into the desired range according to the function. ReLU (Rectified Linear Unit) is most commonly used activation function which converts all negative values into 0 and keep positive values as they are. After conventional and pooling layers, classifier is applied to find probabilities of classes and loss is calculated using loss functions to back propagate the weights. And optimizers are used to select weights after back propagation which has less loss.

It proposes 3D multipath architecture which uses VGG-16 like structure of convolutional and pooling layers. After comparing other architectures, VGG[6] Net is chosen as it is light weight and trains faster. It has VGG-16 structure and fully convolutional layers from different paths, which are concatenated for final output. Lung nodules and malignancy level classifications are done at last layer using SoftMax classifier. Adam optimizer is used to optimize the weights selection for convolutional kernel. Predictions from this architecture are further combined with segmentation model.

Segmentation of lung nodules is additional part adapted from (Julian De Wit, 2017). Before segmentation, lung CT scan images are pre-processed using image processing techniques. Using U-Net[7] architecture as shown in Fig. 4 segmentation masses are generated for lung CT scan images and lung nodules are segmented. Using this model we get another approach of lung nodule detection. After segmentation and classification all results from all models are combined to have more accurate results for lung nodule detection and malignancy prediction.

1.2 Problem Statement

Lung cancer is a type of cancer that begins in the lungs. Your lungs are two spongy organs in your chest that take in oxygen when you inhale and release carbon dioxide when you exhale. Lung cancer is the leading cause of cancer deaths worldwide. People who smoke have the greatest risk of lung cancer, though lung cancer can also occur in people who have never smoked. There are 2 main types of lung cancer and they are treated very differently.

Non-small cell lung cancer (NSCLC)

About 80% to 85% of lung cancers are NSCLC. The main subtypes of NSCLC are adenocarcinoma, squamous cell carcinoma, and large cell carcinoma. These subtypes, which start from different types of lung cells are grouped together as NSCLC because their treatment and prognoses (outlook) are often similar.

Adenocarcinoma: Adenocarcinomas start in the cells that would normally secrete substances such as mucus. This type of lung cancer occurs mainly in people who currently smoke or formerly smoked, but it is also the most common type of lung cancer seen in people who don't smoke. It is more common in women than in men, and it is more likely to occur in younger people than other types of lung cancer.

Adenocarcinoma is usually found in the outer parts of the lung and is more likely to be found before it has spread.

Squamous cell carcinoma: Squamous cell carcinomas start in squamous cells, which are flat cells that line the inside of the airways in the lungs. They are often linked to a history of smoking and tend to be found in the central part of the lungs, near a main airway (bronchus).

Large cell (undifferentiated) carcinoma: Large cell carcinoma can appear in any part of the lung. It tends to grow and spread quickly, which can make it harder to treat. A subtype of large cell carcinoma, known as large cell neuroendocrine carcinoma, is a fast-growing cancer that is very similar to small cell lung cancer.

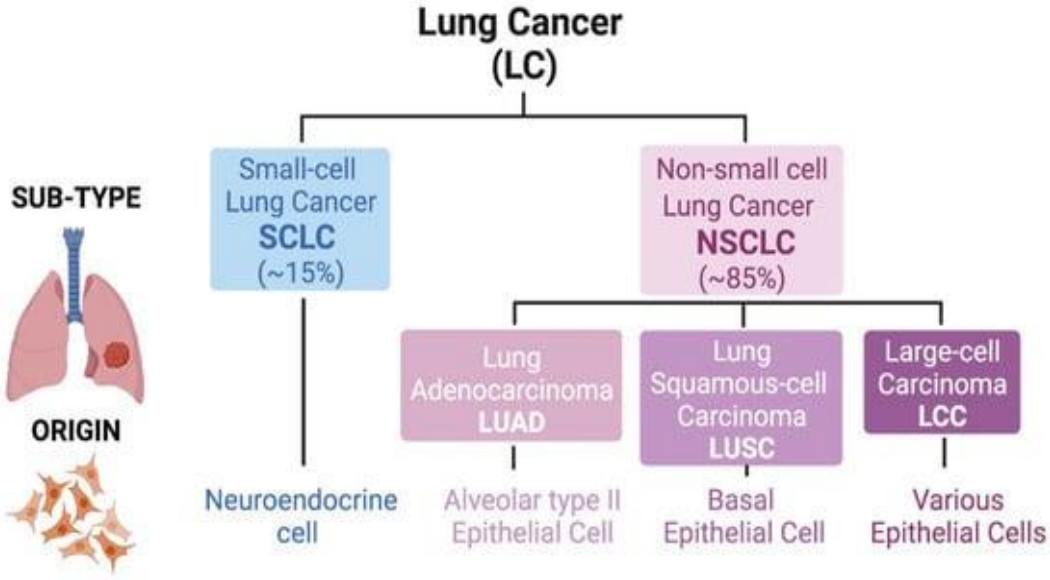


Fig 1.2 Types of Lung Cancer

The primary goal of our project is to diagnose the presence of lung cancer cells based on attributes, which are set of human symptoms, and information. The study explores the possibility of using an Artificial Neural Network model to detect the presence of a lung cancer in someone's body.

1.3 Project Overview

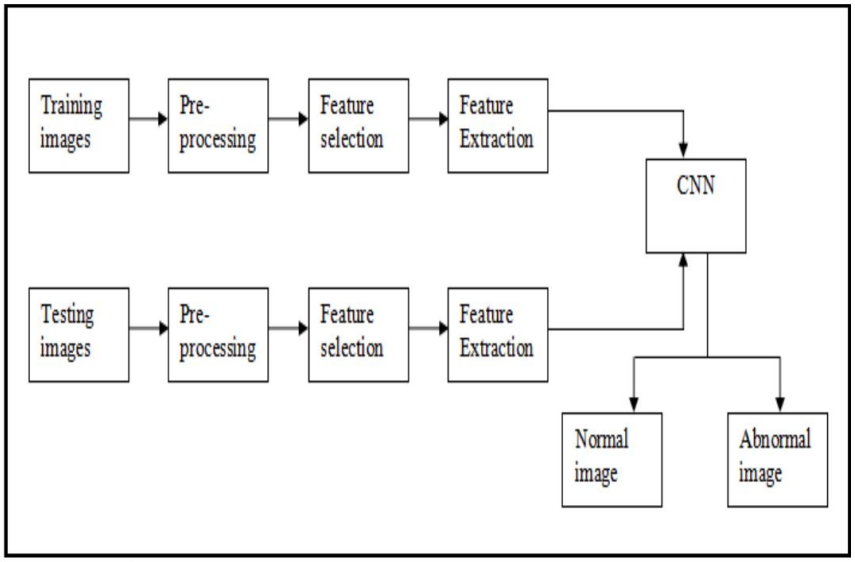
Our Goal is to develop a model which detects cancer in a lung without any user interactions based on Deep Learning and Neural Networks Concepts which helps doctors to efficiently identify the type of Lung Cancer present.

Changes on current best solution have been made and new model has been proposed. Instead of Gabor Filter, Median filter and Gaussian filter have been implemented in pre-processing stage. After preprocessing the processed image is segmented using watershed segmentation. This gives the image with cancer nodules marked. In addition to features like area, perimeter and eccentricity, features like Centroid, Diameter and pixel Mean Intensity have been extracted in feature extraction stage for the detected cancer nodules. The best model ends after the detection of cancer nodule, it's feature extraction and calculation of accuracy. But, its classification as benign or malignant has not been implemented. Therefore, additional stage of classification of cancer nodule has been performed using Deep Learning Algorithms. Extracted features are used as training features and trained model is generated. Then, unknown detected cancer nodule is classified using that trained prediction model.

1.4 Related Work

Sharma and Jindal have suggested a method for identifying lung cancer tumor by using fuzzy interference system and the active contour model. This device makes use of gray level transformation to improve the image contrast. Binarization of images is achieved prior to segmentation, and the resulting image is divided by using active contour model. Moreover, the system efficiency is 94.12 percent. Bhatnagar et al., devised a technique utilizing watershed segmentation. It utilizes Gabor filter in preprocessing to improve the image quality. They compared the effectiveness of the algorithm with neural fuzzy and the region growing method. The accuracy was observed as 89.1 percent which is significantly greater than the method of segmentation using neural fuzzy and region growing method. This framework being the current best solution, but it has certain drawbacks. Just a few

characteristics of cancer nodules have been identified. No preprocessing features such as noise reduction and image smoothing has been applied which will potentially help improve the effective identification of nodules. There has been no proper classification method as benign or malignant. This approach varies from the current best solution, and proposed new method. Median filter and Gaussian filter is introduced in preprocessing stage rather



than Gabor Filter. The best system ends after cancer nodule detection, extraction of feature and accuracy estimation. Yet, it has not enforced its classification as normal or irregular.

Fig 1.3 Image Classification Methodology

Convolutional neural networks are designed for minimizing the number of parameters and adjusting the architecture of the network for image classification. Convolutional neural networks are made up of a series of layers organized according to their features and functionality. A ConvNet 's architecture is close to that of the human brain connectivity pattern of neurons inspired by the Visual Cortex structure. Data augmentation is the process by which data quantity and complexity increase. We will collect fresh data rather than converting the data already available. Data augmentation is an important phase in deep learning, since we require vast quantities of data in deep learning and in certain instances it's not really possible to capture thousands or even millions of images, so this data augmentation comes to the scene. It allows us to maximize the dataset size and add uncertainty within the datasets. In addition to this, webpage is created to store the database of the patients, here the patients can login to their page at any time for the future references. Hospital locality server details can also be inferred easily.

CHAPTER 2

BACKGROUND OF THE PROJECT

2.1 Deep Learning

Deep learning can be considered as a subset of machine learning. It is a field that is based on learning and improving on its own by examining computer algorithms. While machine learning uses simpler concepts, deep learning works with artificial neural networks, which are designed to imitate how humans think and learn. Until recently, neural networks were limited by computing power and thus were limited in complexity. However, advancements in Big Data analytics have permitted larger, sophisticated neural networks, allowing computers to observe, learn, and react to complex situations faster than humans. Deep learning has aided image classification, language translation, speech recognition. It can be used to solve any pattern recognition problem and without human intervention.

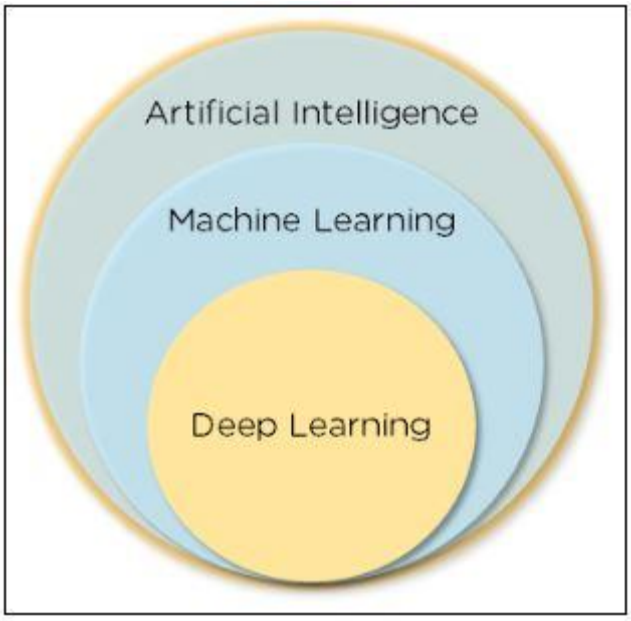


Fig 2.1 Deep Learning as subset of Machine Learning and Artificial Intelligence

Artificial neural networks, comprising many layers, drive deep learning. Deep Neural Networks (DNNs) are such types of networks where each layer can perform complex operations such as representation and abstraction that make sense of images, sound, and text. Considered the fastest-growing field in machine learning, deep learning represents a truly disruptive digital technology, and it is being used by increasingly more companies to create new business models.

Neural networks are layers of nodes, much like the human brain is made up of neurons. Nodes within individual layers are connected to adjacent layers. The network is said to be deeper based on the number of layers it has. A single neuron in the human brain receives thousands of signals from other neurons. In an artificial neural network, signals travel between nodes and assign corresponding weights. A heavier weighted node will exert more effect on the next layer of nodes. The final layer compiles the weighted inputs to produce an output. Deep learning systems require powerful hardware because they have a large amount of data being processed and involves several complex mathematical calculations. Even with such advanced hardware, however, training a neural network can take weeks.

Deep learning systems require large amounts of data to return accurate results; accordingly, information is fed as huge data sets. When processing the data, artificial neural networks are able to classify data with the answers received from a series of binary true or false questions involving highly complex mathematical calculations. For example, a facial recognition program works by learning to detect and recognize edges and lines of faces, then more significant parts of the faces, and, finally, the overall representations of faces. Over time, the program trains itself, and the probability of correct answers increases. In this case, the facial recognition program will accurately identify faces with time.

Importance of Deep Learning-

1- Machine learning works only with sets of structured and semi-structured data, while deep learning works with both structured and unstructured data
2- Deep learning algorithms can perform complex operations efficiently, while machine learning algorithms cannot
3- Machine learning algorithms use labeled sample data to extract patterns, while deep learning accepts large volumes of data as input and analyzes the input data to extract features out of an object
4- The performance of machine learning algorithms decreases as the number of data increases; so to maintain the performance of the model, we need a deep learning

Application of Deep Learning-

Deep learning is widely used to make weather predictions about rain, earthquakes, and tsunamis. It helps in taking the necessary precautions. With deep learning, machines can comprehend speech and provide the required output. It enables the machines to recognize people and objects in the images fed to it.

2.2 Transfer Learning

The reuse of a pre-trained model on a new problem is known as transfer learning in machine learning. A machine uses the knowledge learned from a prior assignment to increase prediction about a new task in transfer learning. You could, for example, use the information gained during training to distinguish beverages when training a classifier to predict whether an image contains cuisine.

The knowledge of an already trained machine learning model is transferred to a different but closely linked problem throughout transfer learning. For example, if you trained a simple classifier to predict whether an image contains a backpack, you could use the model's training knowledge to identify other objects such as sunglasses.

With transfer learning, we basically try to use what we've learned in one task to better understand the concepts in another. weights are being automatically being shifted to a network performing "task A" from a network that performed new "task B."Because of the massive amount of CPU power required, transfer learning is typically applied in computer vision and natural language processing tasks like sentiment analysis.

In computer vision, neural networks typically aim to detect edges in the first layer, forms in the middle layer, and task-specific features in the latter layers. The early and central layers are employed in transfer learning, and the latter layers are only retrained. It makes use of the labelled data from the task it was trained on.

Transfer learning[8] offers a number of advantages, the most important of which are reduced training time, improved neural network performance (in most circumstances), and the absence of a Large Amount of Data.

To train a neural model from scratch, a lot of data is typically needed, but access to that data isn't always possible – this is when transfer learning comes in handy.

Because the model has already been pre-trained, a good machine learning model can be generated with fairly little training data using transfer learning. This is especially useful in natural language processing, where huge labelled datasets require a lot of expert knowledge. Additionally, training time is decreased because building a deep neural network from the start of a complex task can take days or even weeks.

Because the model has already been pre-trained, a good machine learning model can be generated with fairly little training data using transfer learning. This is especially useful in natural language processing, where huge labelled datasets require a lot of expert knowledge. Additionally, training time is decreased because building a deep neural network from the start of a complex task can take days or even weeks.

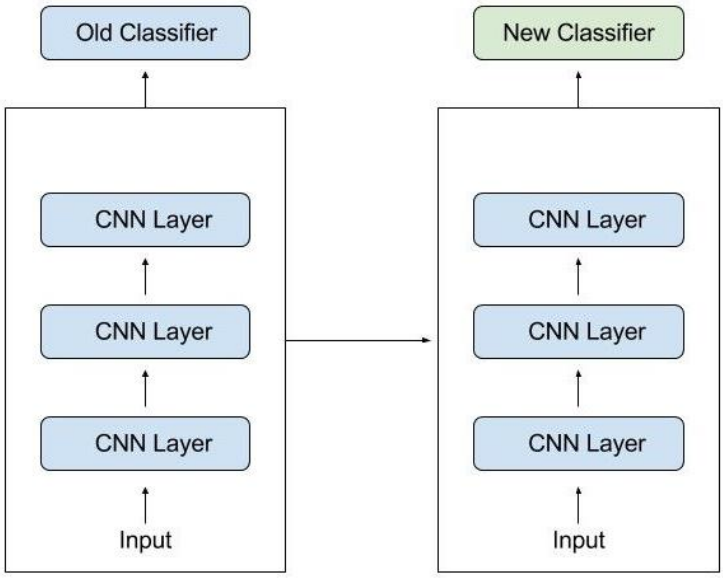


Fig 2.2 Working of Neural Network

2.3 Models of Transfer Learning

2.3.1 EfficientNetB3 Algorithm-

EfficientNet is a convolutional neural network architecture and scaling method that uniformly scales all dimensions of depth/width/resolution using a *compound coefficient*. Unlike conventional practice that arbitrary scales these factors, the EfficientNet scaling method uniformly scales network width, depth, and resolution with a set of fixed scaling coefficients.In general, the EfficientNet models achieve both higher accuracy and better efficiency over existing CNNs, reducing parameter size and FLOPS by an order of magnitude. This function returns a Keras image classification model, optionally loaded with weights pre-trained on ImageNet.

Note: each Keras Application expects a specific kind of input reprocessing. For EfficientNet, input pre-processing is included as part of the model (as a Rescaling layer), and thus tf.keras.applications.efficientnet.preprocess_input is actually a pass-through function.

EfficientNet models expect their inputs to be float tensors of pixels with values in the [0-255] range.

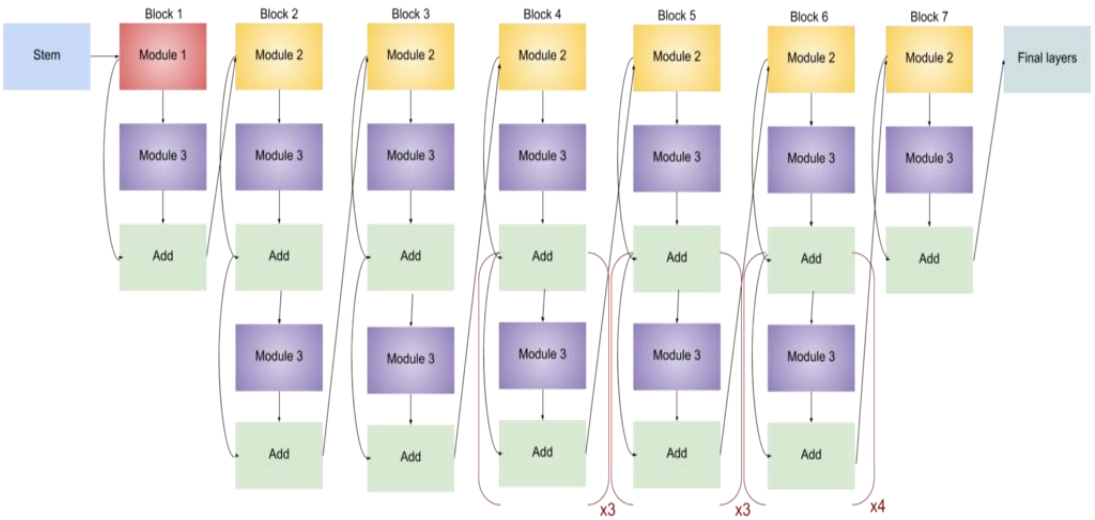


Fig 2.3 EfficientNet Architecture

Arguments-

- **include_top**: Whether to include the fully-connected layer at the top of the network. Defaults to True.
- **weights**: One of None (random initialization), 'imagenet' (pre-training on ImageNet), or the path to the weights file to be loaded. Defaults to 'imagenet'.
- **input_tensor**: Optional Keras tensor (i.e. output of layers.Input()) to use as image input for the model.
- **input_shape**: Optional shape tuple, only to be specified if include_top is False. It should have exactly 3 inputs channels.
- **pooling**: Optional pooling mode for feature extraction when include_top is False. Defaults to None. - None means that the output of the model will be the 4D tensor output of the last convolutional layer. - avg means that global average pooling will be applied to the output of the last convolutional layer, and thus the output of the model will be a 2D tensor. - max means that global max pooling will be applied.
- **classes**: Optional number of classes to classify images into, only to be specified if include_top is True, and if no weights argument is specified. Defaults to 1000 (number of ImageNet classes).

- **classifier_activation**: A str or callable. The activation function to use on the "top" layer. Ignored unless include_top=True. Set classifier_activation=None to return the logits of the "top" layer. Defaults to 'softmax'. When loading pretrained weights, classifier_activation can only be None or "softmax".

Returns-

A keras.Model instance.

2.3.2 DenseNet 169 Algorithm-

It is a type of convolutional neural network that utilises dense connections between layers, through Dense Blocks, where we connect all layers (with matching feature-map sizes) directly with each other**.**

In a DenseNet[10] architecture, each layer is connected to every other layer, hence the name Densely Connected Convolutional Network. For L layers, there are L(L+1)/2 direct connections. For each layer, the feature maps of all the preceding layers are used as inputs, and its own feature maps are used as input for each subsequent layer.

DenseNets essentially connect every layer to every other layer. This is the main idea that is extremely powerful. The input of a layer inside DenseNet is the concatenation of feature maps from previous layers.

Optionally loads weights pre-trained on ImageNet. Note that the data format convention used by the model is the one specified in your Keras config at ~/.keras/keras.json.

Note: each Keras Application expects a specific kind of input pre-processing. For DenseNet, call tf.keras.applications.densenet.preprocess_input on your inputs before passing them to the model.

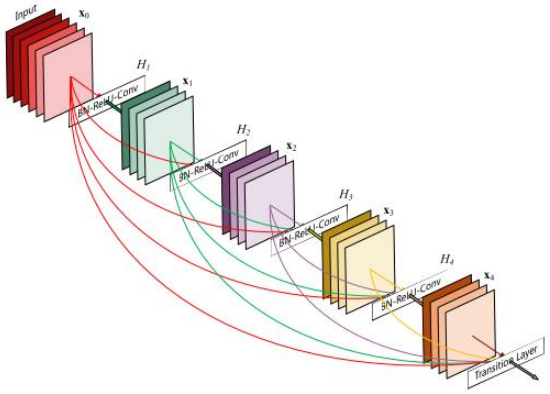


Fig 2.4 DenseNet Architecture

Arguments-

- **include_top**: whether to include the fully-connected layer at the top of the network.
- **weights**: one of None (random initialization), 'imagenet' (pre-training on ImageNet), or the path to the weights file to be loaded.
- **input_tensor**: optional Keras tensor (i.e. output of layers.Input()) to use as image input for the model.
- **input_shape**: optional shape tuple, only to be specified if include_top is False (otherwise the input shape has to be (224, 224, 3) (with 'channels_last' data format) or (3, 224, 224) (with 'channels_first' data format). It should have exactly 3 inputs channels, and width and height should be no smaller than 32. E.g. (200, 200, 3) would be one valid value.
- **pooling**: Optional pooling mode for feature extraction when include_top is False.
    - None means that the output of the model will be the 4D tensor output of the last convolutional block.
    - avg means that global average pooling will be applied to the output of the last convolutional block, and thus the output of the model will be a 2D tensor.
- **classes**: optional number of classes to classify images into, only to be specified if include_top is True, and if no weights argument is specified.
- **classifier_activation**: A str or callable. The activation function to use on the "top" layer. Ignored unless include_top=True. Set classifier_activation=None to return the logits of the "top" layer. When loading pretrained weights, classifier_activation can only be None or "softmax".

Returns-

A Keras model instance.

2.3.3 ResNet 152-

It is an artificial neural network that introduced a so-called "identity shortcut connection," which allows the model to skip one or more layers. This approach makes it possible to train the network on thousands of layers without affecting performance. It's become one of the most popular architectures for various computer vision tasks.

ResNet[9] can have a very deep network of up to 152 layers by learning the residual representation functions instead of learning the signal representation directly. ResNet introduces skip connection (or shortcut connection) to fit the input from the previous layer to the next layer without any modification of the input.

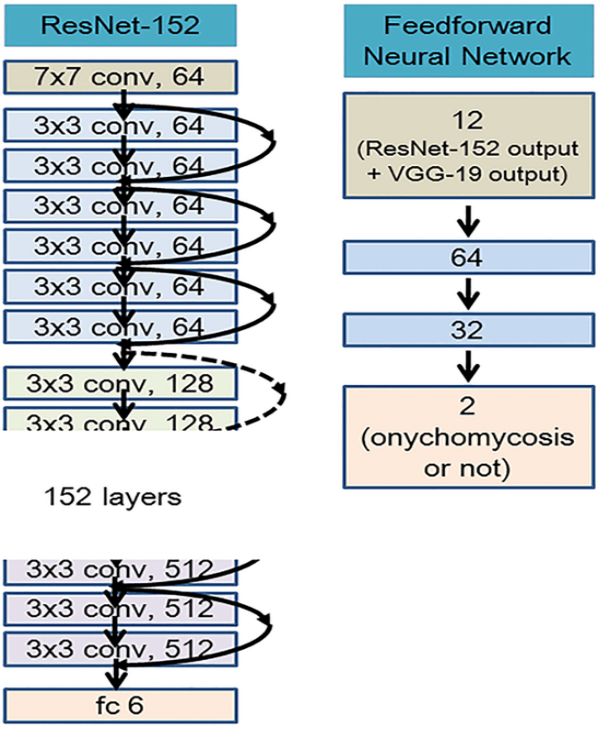


Fig 2.5 ResNet Architecture

Note: each Keras Application expects a specific kind of input preprocessing. For ResNet, call tf.keras.applications.resnet.preprocess_input on your inputs before passing them to the model. resnet.preprocess_input will convert the input images from

RGB to BGR, then will zero-center each color channel with respect to the ImageNet dataset, without scaling.

Arguments-

- **include_top**: whether to include the fully-connected layer at the top of the network.
- **weights**: one of None (random initialization), 'imagenet' (pre-training on ImageNet), or the path to the weights file to be loaded.
- **input_tensor**: optional Keras tensor (i.e. output of layers.Input()) to use as image input for the model.
- **input_shape**: optional shape tuple, only to be specified if include_top is False (otherwise the input shape has to be (224, 224, 3) (with 'channels_last' data format) or (3, 224, 224) (with 'channels_first' data format). It should have exactly 3 inputs channels, and width and height should be no smaller than 32. E.g. (200, 200, 3) would be one valid value.
- **pooling**: Optional pooling mode for feature extraction when include_top is False.
    - None means that the output of the model will be the 4D tensor output of the last convolutional block.
    - avg means that global average pooling will be applied to the output of the last convolutional block, and thus the output of the model will be a 2D tensor.
    - max means that global max pooling will be applied.
- **classes**: optional number of classes to classify images into, only to be specified if include_top is True, and if no weights argument is specified.
- **classifier_activation**: A str or callable. The activation function to use on the "top" layer. Ignored unless include_top=True. Set classifier_activation=None to return the logits of the "top" layer. When loading pretrained weights, classifier_activation can only be None or "softmax".

Returns-

A Keras model instance.

2.3.4 VGG 19-

It is a convolutional neural network that is 19 layers deep. You can load a pretrained version of the network trained on more than a million images from the ImageNet database. The pretrained network can classify images into 1000 object categories, such as keyboard, mouse, pencil, and many animals. As a result, the network has learned rich feature representations for a wide range of images. The network has an image input size of 224-by-224.

VGG stands for Visual Geometry Group (a group of researchers at Oxford who developed this architecture). The VGG architecture consists of blocks, where each block is composed of 2D Convolution and Max Pooling layers.

Note: each Keras Application expects a specific kind of input preprocessing. For VGG19, call tf.keras.applications.vgg19.preprocess_input on your inputs before passing them to the model. vgg19.preprocess_input will convert the input images from RGB to BGR, then will zero-center each color channel with respect to the ImageNet dataset, without scaling.

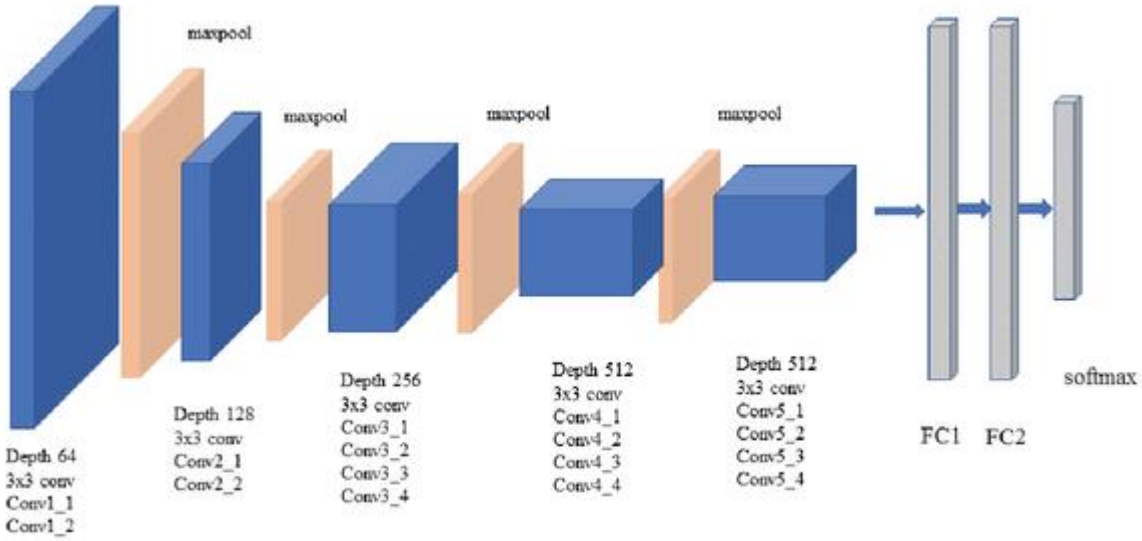


Fig 2.6 VGG Architecture

Arguments-

- **include_top**: whether to include the 3 fully-connected layers at the top of network.
- **weights**: one of None (random initialization), 'imagenet' (pre-training on ImageNet), or the path to the weights file to be loaded.

- **input_tensor**: optional Keras tensor (i.e. output of layers.Input()) to use as image input for the model.
- **input_shape**: optional shape tuple, only to be specified if include_top is False (otherwise the input shape has to be (224, 224, 3) (with channels_last data format) or (3, 224, 224) (with channels_first data format). It should have exactly 3 inputs channels, and width and height should be no smaller than 32. E.g. (200, 200, 3) would be one valid value.
- **pooling**: Optional pooling mode for feature extraction when include_top is False.
    - None means that the output of the model will be the 4D tensor output of the last convolutional block.
    - avg means that global average pooling will be applied to the output of the last convolutional block, and thus the output of the model will be a 2D tensor.
    - max means that global max pooling will be applied.
- **classes**: optional number of classes to classify images into, only to be specified if include_top is True, and if no weights argument is specified.
- **classifier_activation**: A str or callable. The activation function to use on the "top" layer. Ignored unless include_top=True. Set classifier_activation=None to return the logits of the "top" layer. When loading pretrained weights, classifier_activation can only be None or "softmax".

Returns-

A keras.Model instance.

2.3.5 MobileNet V2-

MobileNetV2 is very similar to the original MobileNet, except that it uses inverted residual blocks with bottlenecking features. It has a drastically lower parameter count than the original MobileNet. MobileNet support any input size greater than 32 x 32, with larger image sizes offering better performance.

This function returns a Keras image classification model, optionally loaded with weights pre-trained on ImageNet.

Note: each Keras Application expects a specific kind of input preprocessing. For MobileNetV2,call tf.keras.applications.mobilenet_v2.preprocess_input on your inputs before

passing them to the model. mobilenet_v2.preprocess_input will scale input pixels between -1 and 1.

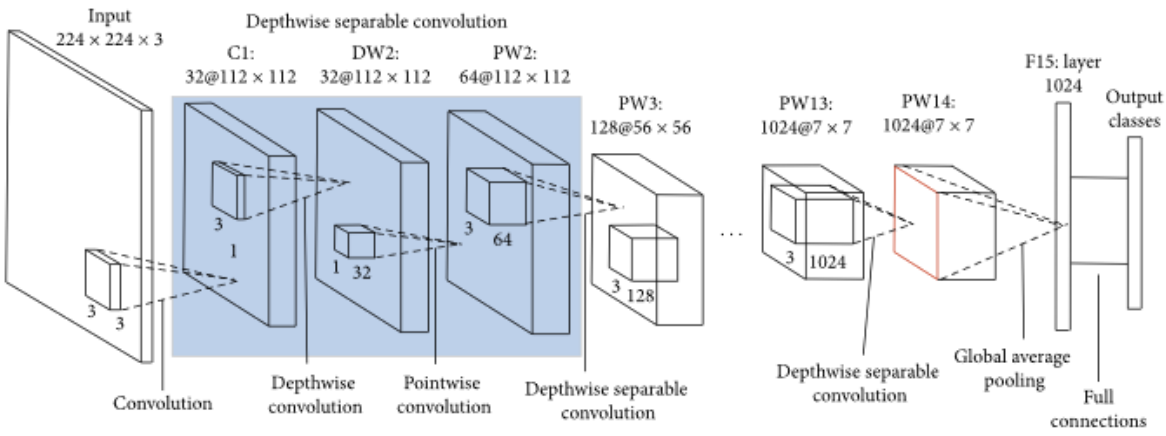


Fig 2.7 Mobile Net Architecture

Arguments-

- **input_shape**: Optional shape tuple, to be specified if you would like to use a model with an input image resolution that is not (224, 224, 3). It should have exactly 3 inputs channels (224, 224, 3). You can also omit this option if you would like to infer input_shape from an input_tensor. If you choose to include both input_tensor and input_shape then input_shape will be used if they match, if the shapes do not match then we will throw an error. E.g. (160, 160, 3) would be one valid value.
- **alpha**: Float, larger than zero, controls the width of the network. This is known as the width multiplier in the MobileNetV2 paper, but the name is kept for consistency with applications.MobileNetV1 model in Keras.
    - If alpha < 1.0, proportionally decreases the number of filters in each layer.
    - If alpha > 1.0, proportionally increases the number of filters in each layer.
    - If alpha = 1.0, default number of filters from the paper are used at each layer.
- **include_top**: Boolean, whether to include the fully-connected layer at the top of the network. Defaults to True.
- **weights**: String, one of None (random initialization), 'imagenet' (pre-training on ImageNet), or the path to the weights file to be loaded.
- **input_tensor**: Optional Keras tensor (i.e. output of layers.Input()) to use as image input for the model.

- **pooling**: String, optional pooling mode for feature extraction when include_top is False.
    - None means that the output of the model will be the 4D tensor output of the last convolutional block.
    - avg means that global average pooling will be applied to the output of the last convolutional block, and thus the output of the model will be a 2D tensor.
- **classes**: Optional integer number of classes to classify images into, only to be specified if include_top is True, and if no weights argument is specified.
- **classifier_activation**: A str or callable. The activation function to use on the "top" layer. Ignored unless include_top=True. Set classifier_activation=None to return the logits of the "top" layer. When loading pretrained weights, classifier_activation can only be None or "softmax".
- ****kwargs**: For backwards compatibility only.

Returns

A keras.Model instance.

2.4 Image Processing

An image is represented by its dimensions (height and width) based on the number of pixels. For example, if the dimensions of an image are 500 x 400 (width x height), the total number of pixels in the image is 200000.

This pixel is a point on the image that takes on a specific shade, opacity or color. It is usually represented in one of the following:

- Grayscale - A pixel is an integer with a value between 0 to 255 (0 is completely black and 255 is completely white).
- RGB - A pixel is made up of 3 integers between 0 to 255 (the integers represent the intensity of red, green, and blue).
- RGBA - It is an extension of RGB with an added alpha field, which represents the opacity of the image.

Image processing requires fixed sequences of operations that are performed at each pixel of an image. The image processor performs the first sequence of operations on the image, pixel

by pixel. Once this is fully done, it will begin to perform the second operation, and so on. The output value of these operations can be computed at any pixel of the image.

Image processing is the process of transforming an image into a digital form and performing certain operations to get some useful information from it. The image processing system usually treats all images as 2D signals when applying certain predetermined signal processing methods.

There are five main types of image processing:

- **Visualization -** Find objects that are not visible in the image
- **Recognition -** Distinguish or detect objects in the image
- **Sharpening and restoration -** Create an enhanced image from the original image
- **Pattern recognition -** Measure the various patterns around the objects in the image
- **Retrieval -** Browse and search images from a large database of digital images that are similar to the original image

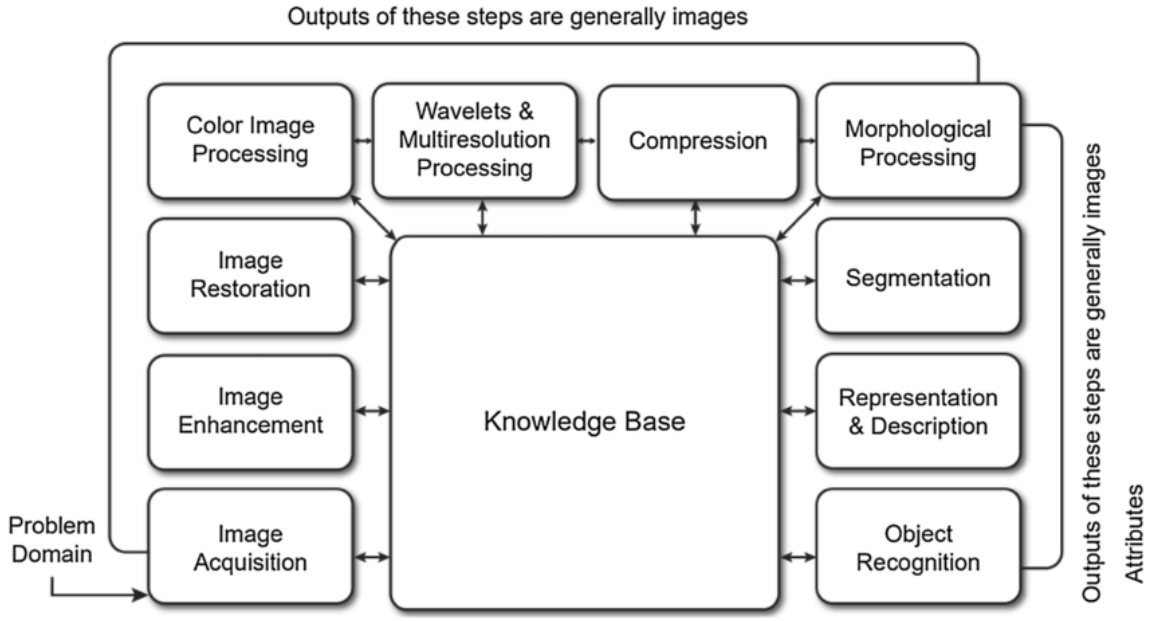


Fig 2.8 Fundamental Steps of Image Processing

Fundamental steps of Image Processing are-

- **Image Acquisition-**Image acquisition is the first step of the fundamental steps of DIP. In this stage, an image is given in the digital form. Generally, in this stage, pre-processing such as scaling is done.
- **Image Enhancement-** Image enhancement is the simplest and most attractive area of DIP. In this stage details which are not known, or we can say that interesting features of an image is highlighted. Such as brightness, contrast, etc...

- **Image Restoration-** Image restoration is the stage in which the appearance of an image is improved.
- **Color Image Processing**- Color image processing is a famous area because it has increased the use of digital images on the internet. This includes color modeling, processing in a digital domain, etc.
- **Wavelets and Multi-Resolution Processing-** In this stage, an image is represented in various degrees of resolution. Image is divided into smaller regions for data compression and for the pyramidal representation.
- **Compression-** Compression is a technique which is used for reducing the requirement of storing an image. It is a very important stage because it is very necessary to compress data for internet use.
- **Morphological Processing-** This stage deals with tools which are used for extracting the components of the image, which is useful in the representation and description of shape.
- **Segmentation-** In this stage, an image is a partitioned into its objects. Segmentation is the most difficult tasks in DIP. It is a process which takes a lot of time for the successful solution of imaging problems which requires objects to identify individually.
- **Representation and Description-** Representation and description follow the output of the segmentation stage. The output is a raw pixel data which has all points of the region itself. To transform the raw data, representation is the only solution. Whereas description is used for extracting information's to differentiate one class of objects from another.
- **Object recognition-** In this stage, the label is assigned to the object, which is based on descriptors.
- **Knowledge Base-** Knowledge is the last stage in DIP. In this stage, important information of the image is located, which limits the searching processes. The knowledge base is very complex when the image database has a high-resolution satellite.

2.5 Histogram Equalization

Histogram Equalization is a computer image processing technique used to improve contrast in images. It accomplishes this by effectively spreading out the most frequent intensity values, i.e. stretching out the intensity range of the image. This method usually increases the global contrast of images when its usable data is represented by close contrast values. This allows for areas of lower local contrast to gain a higher contrast.

It can be classified into two branches as per the transformation function is used.

- **Global histogram equalization (GHE)**

  GHE is very simple and fast, but its contrast enhancement power is low. In this process histogram is flattened and stretched hence improving the overall contrast. is improved.

- **Local histogram equalization (LHE)**

  LHE can enhance the overall contrast more effectively.

The library used for Histogram Equalization in the project is CV2 (Open CV).

OpenCV (Open-Source Computer Vision Library) is an open-source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products. Being an Apache 2 licensed product, OpenCV makes it easy for businesses to utilize and modify the code.

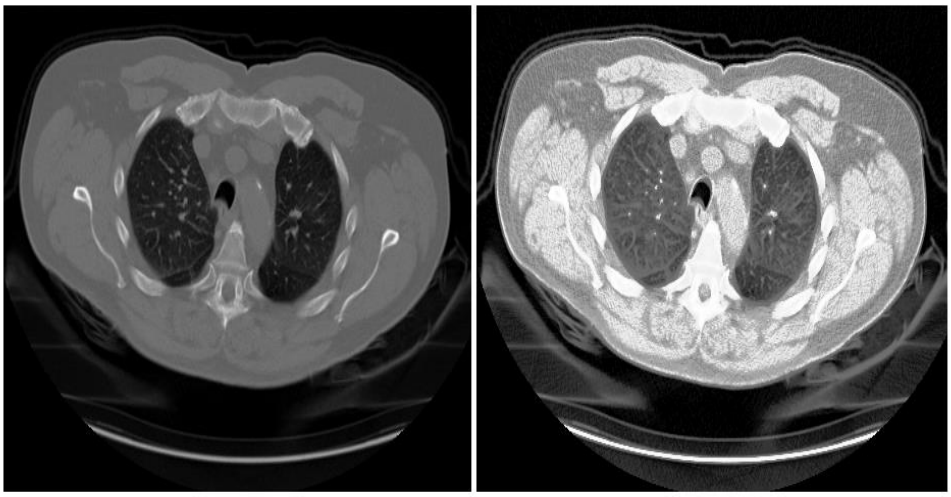


Fig 2.9 Difference between Normal and Equalized Image

The Open CV library has more than 2500 optimized algorithms, which includes a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms. These algorithms can be used to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects, extract 3D models of objects etc.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Designing Steps

Deep learning is a sub-field of machine learning which works on concepts of artificial neural networks to perform specific tasks. Artificial neural networks withdraw inspiration from the human brain.

However, it is paramount to note that they do not function theoretically like our brains, not even close! They are named as artificial neural networks as they can complete precise tasks while achieving a desirable accuracy without being explicitly programmed with any specific rules.

The main reason for the failure of AI a few decades ago was due to the lack of data and computation power. However, this has changed significantly over the past few years. The abundance of data is surging every day because big tech giants and multi-national companies are investing in this data. The computational power is also no longer such a big issue due to powerful graphics processing units (GPUs).

The five essential stages of deep learning models are-

1. Defining Your Architecture —

Deep learning is one of the most preferable methods to solve complex tasks like image classification or segmentation, face recognition, object detection, chatbots, and so much more. But, with each of these projects, every deep learning model goes through five fixed stages to accomplish the task at hand.

The first and most significant step of building your deep learning model is to define the network and architecture successfully. Depending on the type of tasks that are being performed, we prefer to use certain types of architecture.

Usually, Convolutional Neural Networks (CNNs) or ConvNets are preferred for computer vision tasks such as image segmentation, image classification, facial recognition, and other similar projects.

While Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs) are preferable for natural language processing and problems related to text data.

In this step, you can also decide the type of model building structure of the entire deep learning architecture. The three main steps for performing this are the Sequential Models, Functional API, or a custom architecture defined by the user. We will discuss each of these methods in further detail in a future article.

2. Compiling Your Model —

With the preferred architecture constructed, we will move on to the next step of the compilation of the built model. The compile step is usually a single line of code in the TensorFlow deep learning framework and can be achieved with the model.compile() function.

The requirement of the compilation step in deep learning is to configure the model for the fitting/training process to be successfully completed. It is during the compile step that some of the critical components of the training procedure is defined for the evaluation procedure.

To mention a few of the necessary parameters, we have the loss, optimizer, and the metrics to be assigned in the following step. The kind of loss is determined by the type of problem we are encountering and what we need to solve. The optimizers are usually adam, RMSprop, or similar optimizers for computations, and the metrics can be accuracy or any other user-defined metrics for analysis.

3. Fit the Model —

After defining the overall architecture and compiling your model, the next logical step is to fit the model on the training dataset. The fit function trains the model for a fixed number of epochs (iterations on a dataset).

The important parameters like the number of epochs for training, the input and output data, validation data, and many others are decided with the help of this function. It is used for computing and calculating these essential features.

The fitting step must be continuously evaluated while the training procedure is underway. It is essential to make sure the model being trained is performing well with improving accuracies as well as a reduction in the overall loss.

It is equally important to consider that the model is not being overfitted in any manner. For this, continuous evaluation with a tool like Tensorboard must be used for analyzing the various graphs and understanding if these models are being overfitted by any chance.

Once the training is completed and analyzed for a fixed number of epochs, we can move ahead to the next step of the evaluation and making predictions with the trained model.

4. Evaluating And Making Predictions —

Evaluation of deep learning models is an extremely significant step to check out if your article is working out as desired. There are chances that the deep learning model that you built might not perform well in real-world applications. Therefore, evaluation of your deep learning models becomes critical.

One main method of evaluating your deep learning models is to ensure that the predictions made by your model on the test data that is split at the beginning of the pre-processing step are considered for the purposes of validating the effectiveness of the trained model.

Apart from the test data, the model must be tested with variable data and random tests as well to see its effectiveness on un-trained data and if its efficiency of performance matches the required commitments.

To explain with a simple example — let us assume that we built a simple face recognition model. Consider the model you have trained with the images and try to evaluate these images with various faces both on the test data and a real-time video reel as well to make sure the trained model is performing well.

5. Deploying the Model —

The deployment stage is the final stage of any model constructed.

Once you have successfully completed building your model, this is an optional step if you want to keep it with yourself or deploy it so that you can target a wider audience.

The methods of deployment vary from deploying it as an application that can be transferred across, or by using the AWS cloud platform provided by amazon for deployment, or by making use of an embedded system.

If you want to deploy something like a security camera, then you can consider using an embedded device like the raspberry pi alongside a camera module for performing this function. Embedded systems with AI are common ways of deploying your IoT projects.

You can also deploy these deep learning models on your website after it is built with either flask, Django, or any other similar framework. Another way to effectively deploy your models is by developing an android or iOS app for smartphone users to reach a wider range of audiences.

3.1.1 Solution Design

The working of lung cancer detection for the new user using deep learning is described in the below flowchart

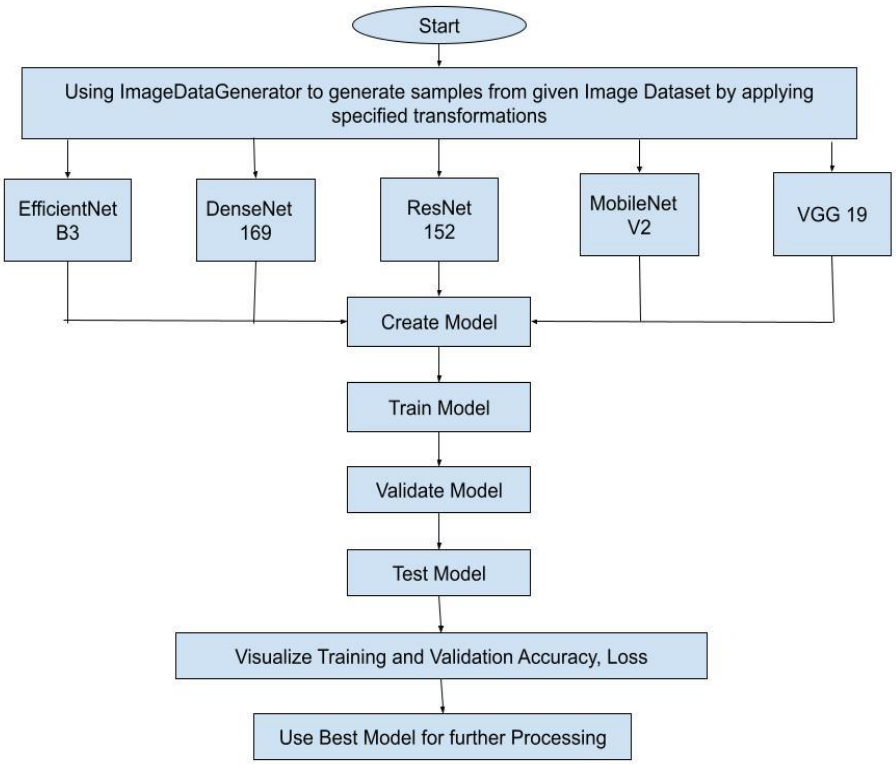


Fig 3.1 Methodology used in the Model

3.1.2 Dataflow Diagram

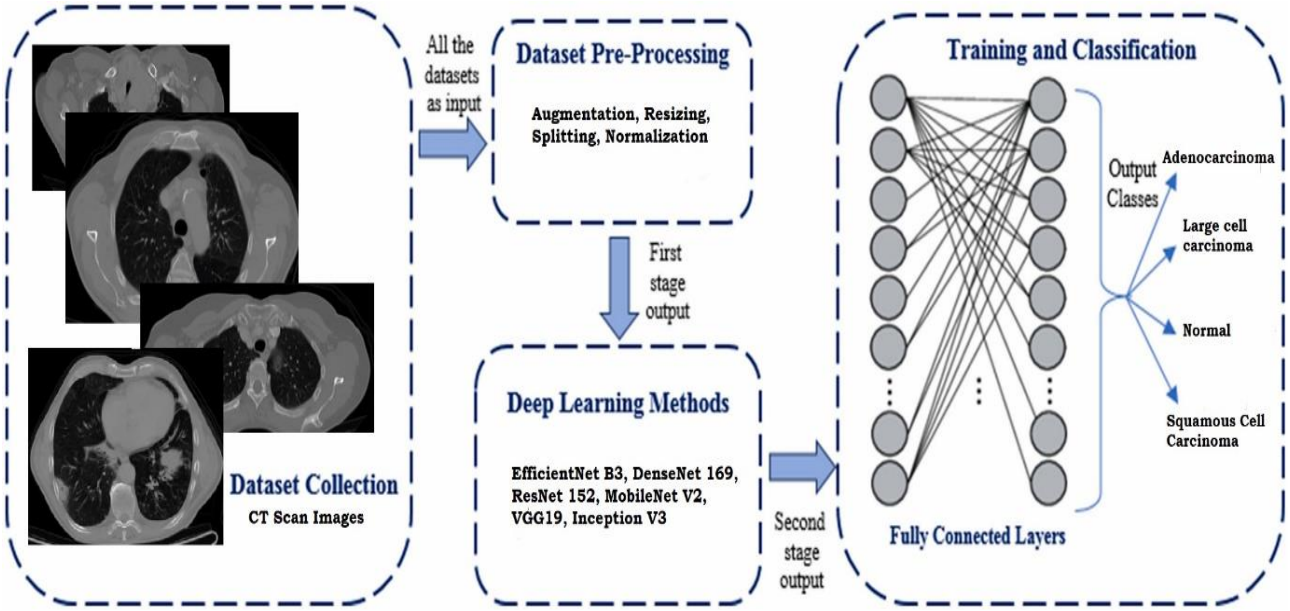


Fig 3.2 Dataflow Diagram of the Model

3.2 Software Specification

Python:

Python is an interpreted, high-level, general purpose programming language created by Guido Van Rossum and first released in 1991, Python's design philosophy emphasizes code Readability with its notable use of significant Whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically typed and garbage collected. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming

PIP:

It is the package management system used to install and manage software packages written.

3.3 Hardware Specification

- **Processor:** Intel core i5 or above
- **Architecture:** 64-bit, Quad Core, 2.5 GHz minimum per core
- **Ram:** 4 GB or More
- **Hard disk**: 10 GB of available space or more
- **Display:** Dual XGA (1024 x 768) or higher resolution Monitors
- **Operating System:** Windows

3.4 Libraries Used

a. NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays. It is the fundamental package for scientific computing with Python. It is open-source software. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code

- Useful linear algebra, Fourier transform, and random number capabilities

NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

b. Pandas

Pandas is an open-source library that is made mainly for working with relational or labeled data both easily and intuitively. It provides various data structures and operations for manipulating numerical data and time series. This library is built on top of the NumPy library. Pandas is fast and it has high performance & productivity for users.

Some Advantages of using Pandas are-

- Fast and efficient for manipulating and analyzing data.
- Data from different file objects can be loaded.
- Easy handling of missing data (represented as NaN) in floating point as well as non-floating point data
- Size mutability: columns can be inserted and deleted from DataFrame and higher dimensional objects
- Data set merging and joining.
- Flexible reshaping and pivoting of data sets
- Provides time-series functionality.
- Powerful group by functionality for performing split-apply-combine operations on data sets.

c. Matplotlib

Matplotlib is an amazing visualization library in Python for 2D plots of arrays. Matplotlib is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack. It was introduced by John Hunter in the year 2002.

One of the greatest benefits of visualization is that it allows us visual access to huge amounts of data in easily digestible visuals. Matplotlib consists of several plots like line, bar, scatter, histogram etc.

d. TensorFlow

TensorFlow is an end-to-end open source platform for machine learning. TensorFlow is a rich system for managing all aspects of a machine learning system; however, this class focuses on using a particular TensorFlow API to develop and train machine learning models.

TensorFlow APIs are arranged hierarchically, with the high-level APIs built on the low-level APIs. Machine learning researchers use the low-level APIs to create and explore new machine learning algorithms. In this class, you will use a high-level API named tf.keras to define and train machine learning models and to make predictions. tf.keras is the TensorFlow variant of the open-source Keras API.

The following figure shows the hierarchy of TensorFlow toolkits:

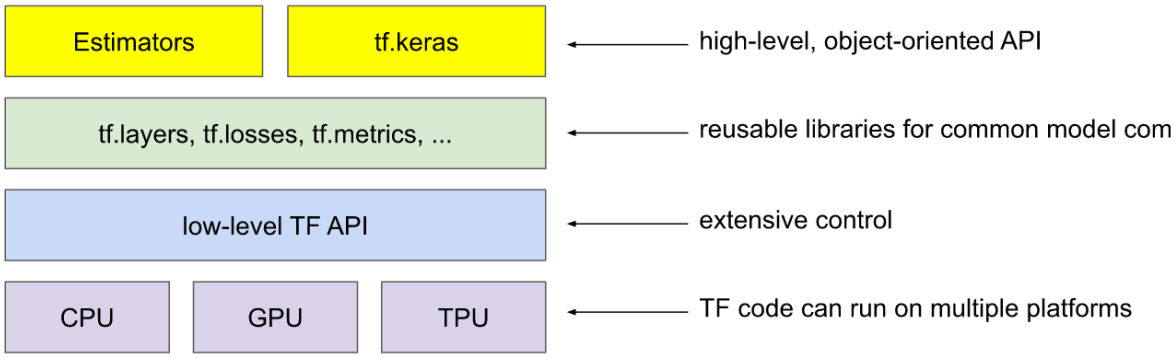


Fig 3.3 Hierarchy of TensorFlow Toolkit

e. Keras

Keras is a high-level, deep learning API developed by Google for implementing neural networks. It is written in Python and is used to make the implementation of neural networks easy. It also supports multiple backend neural network computation.

Keras is relatively easy to learn and work with because it provides a python frontend with a high level of abstraction while having the option of multiple back-ends for computation purposes. This makes Keras slower than other deep learning frameworks, but extremely beginner-friendly.

Keras allows you to switch between different back ends. The frameworks supported by Keras are:

- TensorFlow
- Theano
- PlaidML

- MXNet
- CNTK (Microsoft Cognitive Toolkit

Out of these five frameworks, TensorFlow has adopted Keras as its official high-level API. Keras is embedded in TensorFlow and can be used to perform deep learning fast as it provides inbuilt modules for all neural network computations. At the same time, computation involving tensors, computation graphs, sessions, etc can be custom made using the TensorFlow Core API, which gives you total flexibility and control over your application and lets you implement your ideas in a relatively short time.

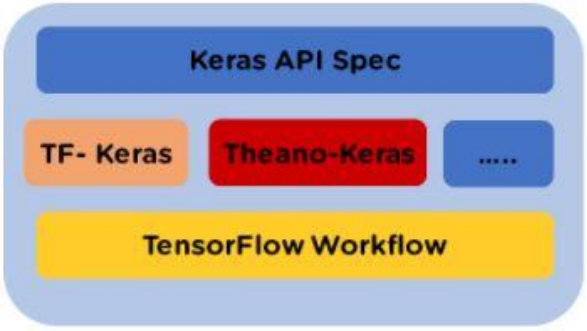


Fig 3.4 Keras API Specification

3.5 Platforms Used

a. Jupyter Notebook

Jupyter Notebook is an **open-source, web-based interactive environment,** which allows you to create and share documents that contain **live code, mathematical equations, graphics, maps, plots, visualizations,** and **narrative text**. It integrates with many programming languages like **Python, PHP, R, C#,** etc.

Advantages of Using Jupyter Notebook are-

- **All in one place:** As you know, Jupyter Notebook is an open-source web-based interactive environment that combines code, text, images, videos, mathematical equations, plots, maps, graphical user interface and widgets to a single document.
- **Easy to convert:** Jupyter Notebook allows users to convert the notebooks into other formats such as HTML and PDF. It also uses online tools and nbviewer which allows you to render a publicly available notebook in the browser directly.
- **Easy to share:** Jupyter Notebooks are saved in the structured text files (JSON format), which makes them easily shareable.
- **Language independent:** Jupyter Notebook is platform-independent because it is represented as JSON (JavaScript Object Notation) format, which is a language-

independent, text-based file format. Another reason is that the notebook can be processed by any programing language, and can be converted to any file formats such as Markdown, HTML, PDF, and others.

- **Interactive code:** Jupyter notebook uses **ipywidgets** packages, which provide many common user interfaces for exploring code and data interactivity.

Disadvantages of Using Jupyter Notebook are-

- It is very hard to test long asynchronous tasks.
- Less Security
- It runs cell out of order
- In Jupyter notebook, there is no IDE integration, no linting, and no code-style correction.

b. Anaconda Navigator

Anaconda is an open-source distribution of the Python and R programming languages for data science that aims to simplify package management and deployment. Package versions in Anaconda are managed by the package management system, conda, which analyzes the current environment before executing an installation to avoid disrupting other frameworks and packages.

The Anaconda distribution comes with over 250 packages automatically installed. Over 7500 additional open-source packages can be installed from PyPI as well as the conda package and virtual environment manager. It also includes a GUI (graphical user interface), Anaconda Navigator, as a graphical alternative to the command line interface. Anaconda Navigator is included in the Anaconda distribution, and allows users to launch applications and manage conda packages, environments and channels without using command-line commands. Navigator can search for packages, install them in an environment, run the packages and update them.

c. Google Colab

Google Colab was developed by Google to provide free access to GPU's and TPU's to anyone who needs them to build a machine learning or deep learning model. Google Colab can be defined as an improved version of Jupyter Notebook.

Some of exciting features offered by Google Colab are-

- Interactive tutorials to learn machine learning and neural networks.
- Write and execute Python 3 code without having a local setup.

- Execute terminal commands from the Notebook.
- Import datasets from external sources such as Kaggle.
- Save your Notebooks to Google Drive.
- Import Notebooks from Google Drive.
- Free cloud service, GPUs and TPUs.
- Integrate with PyTorch, Tensor Flow, Open CV.
- Import or publish directly from/to GitHub.

3.6 Guiding Principles

- User friendliness: Keras is an API designed for human beings, not machines.Keras offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear and actionable feedback upon user error.
- Modularity: A model is understood as a sequence or a graph of standalone. In particular, neural layers, cost functions, optimizers, initialization schemes, activation functions and regularization schemes are all standalone modules that you can combine to create new models.
- Easy extensibility: New modules are simple to add and existing modules provide ample examples. To be able to easily create new modules allows for total expressiveness, making Keras suitable for advanced research.
- Work with Python: No separate models configuration files in a declarative format. Models are described in Python code, which is compact, easier to debug, and allows for ease of extensibility.

CHAPTER 4

LITERATURE SURVEY

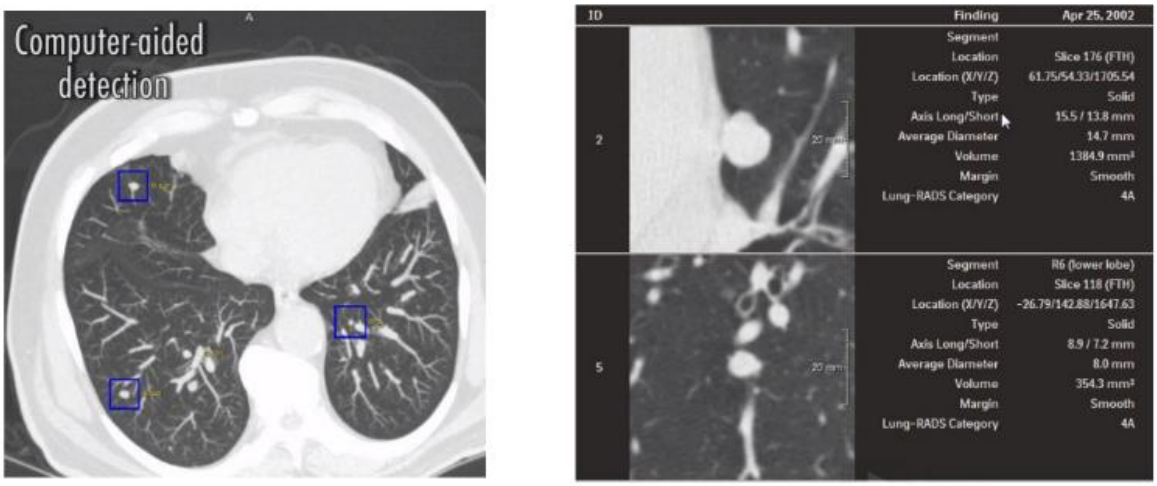
4.1 Existing System

According to Mayo Clinic, In order to diagnose lung cancer, The recommended ways are:

• Imaging tests where an CT Scan on your lungs reveal abnormal mass or nodules.

• Sputum Cytology which analysing Phlegm (Mucus Cells).

• Biopsy which is the most invasive method where abnormal cells are removed from your body to be analysed. This method is done in a number of ways such as bronchoscopy in which an incision is made at your neck and surgical tools are inserted behind your breastbone to take tissue samples**.**



(a) CADe: Computer-Aided Detection System (b) CADx: Computer-Aided Diagnosis Systems

Fig 4.1 Computer Aided Detection (CAD)

The research on current tests on medical imaging for lung cancer detection has been undertaken. There exists computer aided detection systems that assist radiologists in detecting anomalies. According to Fermino et al[23], There are two main computational systems developed to assist radiologists, they are: CADe(Computer-Aided detection system)Figure 2.2a and CADx(computer-aided diagnosis system).Figure 2.2b. CADe detects abnormal mass in medical images while CADx aims to measure the characteristics of the once it has been detected.

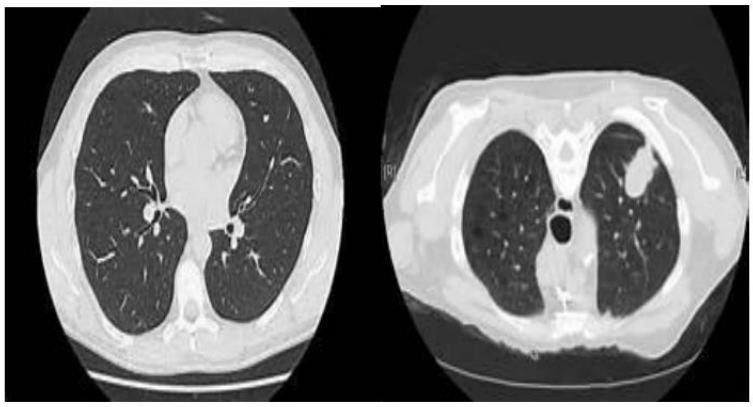
4.2 Proposed System

These square measure steps projected within the system: - These steps can facilitate America police investigation the pre stages for early detection of carcinoma, this will facilitate America to urge the simplest accuracy through image process in these, phase the image and supported that we have a tendency to are going to categories them to the process stage so comparison it with the past pictures of respiratory organ which is able to facilitate in police investigation

cancer. This will facilitate individuals from reducing the risks of police investigation cancer within the last stages whereas doing a pre-test of cancer detection. These projects can facilitate individuals worldwide and from being late or delay in police investigation cancer. Pre-processing, Scanning, Training, Testing and confirmatory as a result of the info needed to coach Deep Learning Models is incredibly giant, it's usually fascinating to coach the model in batches. Loading all the coaching information into memory isn't invariably potential as a result of you would like enough memory to handle it and therefore the options too. I made a decision to load all pictures into a hdf's dataset exploitation h5py library.

Adenocarcinoma is one of the most cancerous diseases which is creating a worldwide problem and increasing the death ratio. So, there are many upcoming projects that will help to reduce the chances and detect them in earlier stage. So, this image is the CT scan image of the lung's cancer, named Adenocarcinoma. We train all the images from the database.

Images of healthy lung to cancerous lung



Healthy lung Cancerous lung

Fig 4.2 Cancerous Lung vs Healthy Lung

4.3 Feasibility Study

From January 2020 to May 2021, a single-center observational cohort study was performed and, prior to investigations, reviewed and accepted by the ethics committee for clinical studies of the Heidelberg University (registry number S-665-2019). The study included (1) patients who gave written informed consent (or their parents if a patient was younger than 18 years); (2) participants were diagnosed as having an OSCC; and (3) resection of the tumor was indicated. The exclusion criteria were: (1) benign pathology of oral mucosa; and (2) scars, previous surgery, or other treatments. Twenty patients with histologically proven OSCC were

selected, and a total of 20 specimens were identified, removed, and described clinically and histopathologically (tumor location, histological grading).

Immediately after resection, the carcinoma excisions were prepared according to the protocol: rinsed in isotonic saline solution, immersed for 20 s in 1 mM acridine orange solution, and rinsed again for approximately 5 s, similar to those already described in other studies [25]. After this, the tissue samples proceeded to ex vivo FCM investigation, which was performed with a Vivascope 2500 Multilaser (Lucid Inc., Rochester, New York, NY, USA) in a combined reflectance and fluorescence mode [25]. The standard imaging of a 2.50 × 2.50 cm tissue sample took, on average, between 40 and 90 s. Hereafter, the samples proceeded to conventional histopathological examinations.

The obtained ex vivo FCM scans of OSCC were annotated according to the WHO criteria: irregular epithelial stratification, the disturbed polarity of the basal cells, disturbed maturational sequence, destruction of the basal membrane and cellular pleomorphism, nuclear hyperchromatism, increase in the nuclear-cytoplasmic ratio, and loss of cellular adhesion and cohesion [26]. Tissue areas that showed clear signs of squamous cell carcinoma were marked using the QuPath bioimaging analysis software. The option to enlarge the tissue samples maintaining high image quality makes a higher precision during the delineation process possible (https://qupath.github.io, accessed on 8 November 2021) [25]. A differentiation between tumor cells and dysplasia was made using different colors. After loading the high-resolution images, the annotation process was conducted. Unclear regions were discussed with an external histopathological specialist. Any regions that did not contain OSCC but included inflammatory or normal tissue were included under the non-neoplastic category. The average annotation time per picture was about 3–4 h. If necessary, the annotations were verified by pathologists. Each annotated image was seen at least by two examiners and one senior histopathologist. Cases with unclear presentations were excluded from training.

The main principle of a Deep Leading model is the application of depth wise separable convolutions. These convolutions separate into a 1 × 1 pointwise convolution with a single filter being applied to each input channel. The pointwise convolution then applies a 1 × 1 convolution to combine the outputs. The depth wise convolution performs a separate application on each channel compared to standard convolutions, where each kernel is applied on all channels of the input image. The model was selected due to its satisfying performances in various computer vision tasks and its feasibility towards real-time applications and transfer learnings for limited datasets. Further, this approach significantly reduces the number of hyperparameters and thus the computational power required for the tasks. Furthermore, it uses

two global hyperparameters, minimizing the need for extensive hyperparameter tuning to obtain an efficient model. The input layer of our models consisted of 256 × 256 × 3 images. After the application of convolutional layers, an 8 × 8 × 512 image output was obtained. Hereafter, a global average pooling was applied, which then resulted in 2 neurons, meaning that there were 512 × 2 weights in addition to 2 biases for each neuron. The output was a binary classification. For input, if more than 50% of the 256 × 256 pixels area is annotated as cancerogenic, then the whole patch is considered as such and is classified as malignant (Figure 2). From 50,000 generated patches, 8000 were considered as malignant and 42,000 as non-malignant

CHAPTER 5

IMPLEMENTATION

5.1 EfficientNetB3

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
==================================================================
import numpy as np
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from keras import Sequential
from tensorflow.keras.layers import *
from tensorflow.keras.models import *
from tensorflow.keras.preprocessing import image
==================================================================
train_datagen = image.ImageDataGenerator(
    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
test_datagen= image.ImageDataGenerator(    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
==================================================================
```

```
train_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/train',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
test_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/test',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
validation_generator = test_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid',
    target_size = (224,224),
    batch_size = 8,
    shuffle=True,
    class_mode = 'categorical')
```

Found 613 images belonging to 4 classes.

Found 315 images belonging to 4 classes.

Found 91 images belonging to 4 classes.

```
==================================================================

base_model=tf.keras.applications.EfficientNetB3(weights='imagenet',input_shape=(224,224,
3), include_top=False)
for layer in base_model.layers:
    layer.trainable=True
model = Sequential()
model.add(base_model)
model.add(GaussianNoise(0.25))
model.add(GlobalAveragePooling2D())
model.add(Dense(1024,activation='relu'))
model.add(BatchNormalization())
model.add(GaussianNoise(0.25))
model.add(Dropout(0.25))
```

```
model.add(Dense(4, activation='sigmoid'))
model.summary()
```

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb3_notop.h5

43941136/43941136 [==============================] - 3s 0us/step

Model: "sequential"

Layer (type)	*Output Shape*	*Param #*
efficientnetb3 (Functional)	*(None, 7, 7, 1536)*	*10783535*
gaussian_noise (GaussianNoise)	*(None, 7, 7, 1536)*	*0*
global_average_pooling2d (GlobalAveragePooling2D)	*(None, 1536)*	*0*
dense (Dense)	*(None, 1024)*	*1573888*
batch_normalization (BatchNormalization)	*(None, 1024)*	*4096*
gaussian_noise_1 (GaussianNoise)	*(None, 1024)*	*0*
dropout (Dropout)	*(None, 1024)*	*0*
dense_1 (Dense)	*(None, 4)*	*4100*

Total params: 12,365,619

Trainable params: 12,276,268

Non-trainable params: 89,351

```
==================================================================
model.compile(loss='categorical_crossentropy',
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
        metrics=['accuracy','AUC','Precision','Recall'])
==================================================================
from keras.callbacks import EarlyStopping
es=EarlyStopping(monitor='val_loss',patience=3)
```

```
history = model.fit(
    train_generator,
    epochs=100,
    validation_data=validation_generator,
    steps_per_epoch= 75  )
```

Epoch 1/100

75/75 [==============================] - 403s 5s/step - loss: 1.9328 - accuracy: 0.2948 - auc: 0.5533 - precision: 0.2696 - recall: 0.5595 - val_loss: 1.3090 - val_accuracy: 0.3626 - val_auc: 0.6449 - val_precision: 0.3403 - val_recall: 0.5385

Epoch 2/100

75/75 [==============================] - 20s 260ms/step - loss: 1.5484 - accuracy: 0.4171 - auc: 0.6483 - precision: 0.3285 - recall: 0.6432 - val_loss: 1.2531 - val_accuracy: 0.3956 - val_auc: 0.6803 - val_precision: 0.3526 - val_recall: 0.6703

....

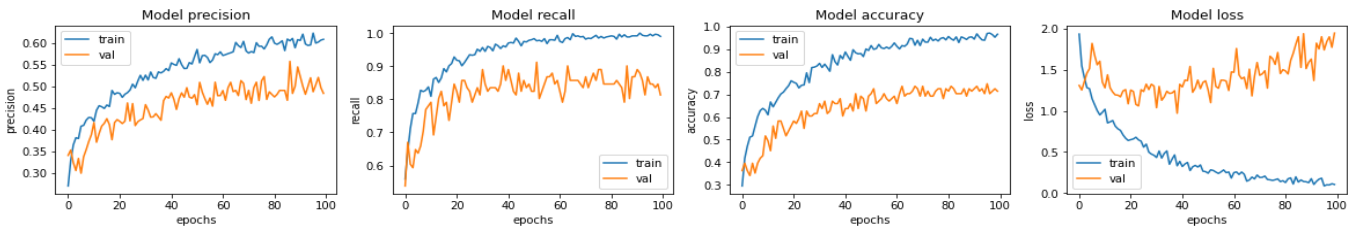
....

Epoch 100/100

75/75 [==============================] - 18s 232ms/step - loss: 0.1057 - accuracy: 0.9665 - auc: 0.9899 - precision: 0.6087 - recall: 0.9899 - val_loss: 1.9448 - val_accuracy: 0.7143 - val_auc: 0.8722 - val_precision: 0.4837 - val_recall: 0.8132

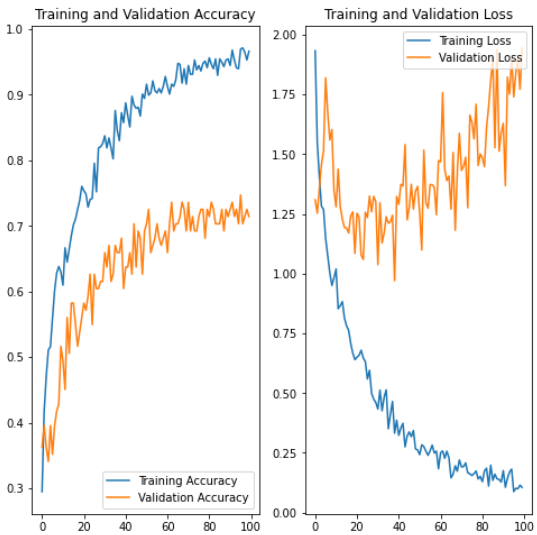
==========

```
fig,ax=plt.subplots(1,4,figsize=(20,3))
ax=ax.ravel()
for i,met in enumerate(['precision','recall','accuracy','loss']):
  ax[i].plot(history.history[met])
  ax[i].plot(history.history['val_'+met])
  ax[i].set_title('Model {}'.format(met))
  ax[i].set_xlabel('epochs')
  ax[i].set_ylabel(met)
  ax[i].legend(['train', 'val'])
```




```
acc = history.history['accuracy']
```

```
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(100)
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```



==========

```
model.evaluate(train_generator)
```

77/77 [==============================] - 11s 139ms/step - loss: 0.0121 - accuracy: 0.9967 - auc: 0.9996 - precision: 0.7503 - recall: 1.0000

[0.01212927233427763,

0.9967373609542847,

0.9995608329772949,

0.7503060102462769,

1.0]

==========

model.evaluate(test_generator)

40/40 [==========] - 217s 6s/step - loss: 0.3842 - accuracy: 0.9016 - auc: 0.9704 - precision: 0.5757 - recall: 0.9778

[0.3841574490070343,

0.9015873074531555,

0.9704425930976868,

0.5757009387016296,

0.9777777791023254]

==========

model.evaluate(validation_generator)

12/12 [==========] - 2s 129ms/step - loss: 2.0475 - accuracy: 0.7363 - auc: 0.8702 - precision: 0.5203 - recall: 0.8462

[2.0475432872772217,

0.7362637519836426,

0.8702250123023987,

0.5202702879905701,

0.8461538553237915]

==========

model.save('/content/drive/MyDrive/ColabNotebooks/FinalYearProject/EfficientNetB3.h5',

model)

==========

from keras.preprocessing import image

img=tf.keras.utils.load_img('/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid/squamous.cell.carcinoma_left.hilum_T1_N2_M0_IIIa/000117 (4).png',target_size=(224,224))

imag = tf.keras.utils.img_to_array(img)

imaga = np.expand_dims(imag,axis=0)

ypred = model.predict(imaga)

print(ypred)

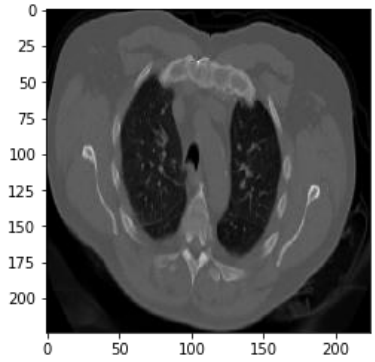
a=np.argmax(ypred,-1)

```
if a==0:
  op="Adenocarcinoma"
elif a==1:
  op="large cell carcinoma"
elif a==2:
  op="normal (void of cancer)"
else:
  op="squamous cell carcinoma"
plt.imshow(img)
print("THE UPLOADED IMAGE IS SUSPECTED AS: "+str(op))
```

1/1 [==============================] - 3s 3s/step

[[0.22290322 0.04368863 0.0361404 0.99998105]]

THE UPLOADED IMAGE IS SUSPECTED AS: squamous cell carcinoma



5.2 DenseNet169

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
==================================================================
import numpy as np
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from keras import Sequential
from tensorflow.keras.layers import *
```

```
from tensorflow.keras.models import *
from tensorflow.keras.preprocessing import image
============================================================
train_datagen = image.ImageDataGenerator(
    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
test_datagen= image.ImageDataGenerator(    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
============================================================
train_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/train',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
test_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/test',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
validation_generator = test_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid',
    target_size = (224,224),
```

```
    batch_size = 8,
    shuffle=True,
    class_mode = 'categorical')
```

Found 613 images belonging to 4 classes.

Found 315 images belonging to 4 classes.

Found 91 images belonging to 4 classes.

```
==================================================================
base_model=tf.keras.applications.DenseNet169(weights='imagenet',input_shape=(224,224,3)
, include_top=False)
for layer in base_model.layers:
    layer.trainable=True
model = Sequential()
model.add(base_model)
model.add(GaussianNoise(0.25))
model.add(GlobalAveragePooling2D())
model.add(Dense(1024,activation='relu'))
model.add(BatchNormalization())
model.add(GaussianNoise(0.25))
model.add(Dropout(0.25))
model.add(Dense(4, activation='sigmoid'))
model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet169_weights_tf_dim_ordering_tf_kernels_notop.h5

51877672/51877672 [==============================] - 4s 0us/step

Model: "sequential"

Layer (type)	*Output Shape*	*Param #*
densenet169 (Functional)	*(None, 7, 7, 1664)*	*12642880*
gaussian_noise (GaussianNoise)	*(None, 7, 7, 1664)*	*0*
global_average_pooling2d	*(G (None, 1664)*	*0*

lobalAveragePooling2D)

dense (Dense)	*(None, 1024)*	*1704960*
batch_normalization (BatchN ormalization)	*(None, 1024)*	*4096*
gaussian_noise_1 (GaussianN oise)	*(None, 1024)*	*0*
dropout (Dropout)	*(None, 1024)*	*0*
dense_1 (Dense)	*(None, 4)*	*4100*

Total params: 14,356,036

Trainable params: 14,195,588

Non-trainable params: 160,448

==========

```
model.compile(loss='categorical_crossentropy',
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
        metrics=['accuracy','AUC','Precision','Recall'])
```

==========

```
from keras.callbacks import EarlyStopping
es=EarlyStopping(monitor='val_loss',patience=3)
history = model.fit(
   train_generator,
   epochs=100,
   validation_data=validation_generator,
   steps_per_epoch= 75   )
```

Epoch 1/100

75/75 [==========] - 390s 5s/step - loss: 1.6236 - accuracy: 0.3719 - auc: 0.6230 - precision: 0.3148 - recall: 0.6365 - val_loss: 1.1567 - val_accuracy: 0.4176 - val_auc: 0.7316 - val_precision: 0.3756 - val_recall: 0.8791

Epoch 2/100

75/75 [==========] - 17s 223ms/step - loss: 1.2082 - accuracy: 0.5461 - auc: 0.7564 - precision: 0.3927 - recall: 0.7755 - val_loss: 1.0885 - val_accuracy: 0.5055 - val_auc: 0.7604 - val_precision: 0.3946 - val_recall: 0.8022

....

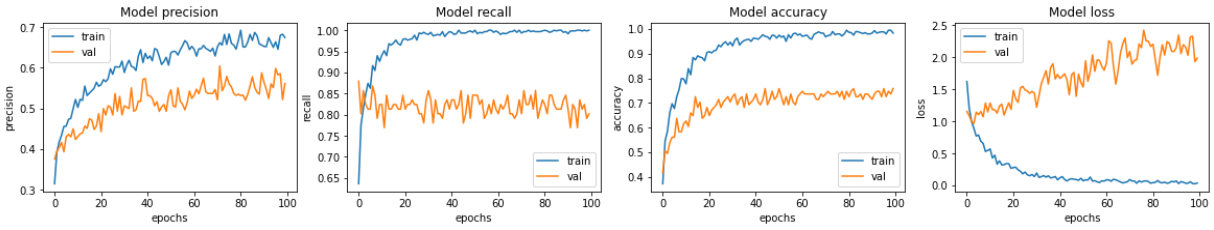
....

Epoch 100/100

75/75 [==============================] - 14s 183ms/step - loss: 0.0364 - accuracy: 0.9832 - auc: 0.9965 - precision: 0.6746 - recall: 1.0000 - val_loss: 1.9881 - val_accuracy: 0.7582 - val_auc: 0.8731 - val_precision: 0.5615 - val_recall: 0.8022

==========

```
fig,ax=plt.subplots(1,4,figsize=(20,3))
ax=ax.ravel()
for i,met in enumerate(['precision','recall','accuracy','loss']):
  ax[i].plot(history.history[met])
  ax[i].plot(history.history['val_'+met])
  ax[i].set_title('Model {}'.format(met))
  ax[i].set_xlabel('epochs')
  ax[i].set_ylabel(met)
  ax[i].legend(['train', 'val'])
```



==========

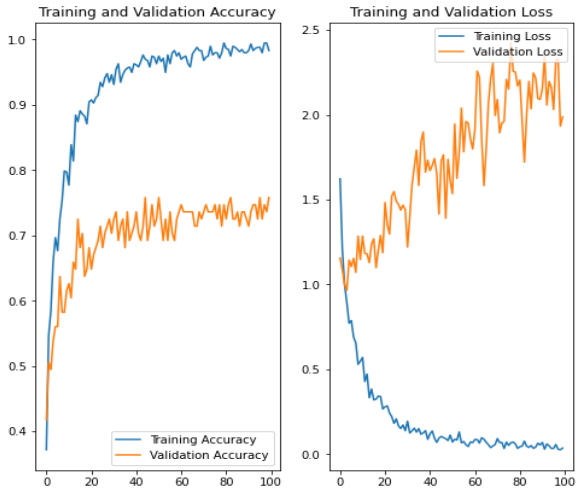
```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(100)
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
```

plt.plot(epochs_range, val_loss, label='Validation Loss')

plt.legend(loc='upper right')

plt.title('Training and Validation Loss')

plt.show()



==========

model.evaluate(train_generator)

77/77 [==========] - 11s 137ms/step - loss: 0.0044 - accuracy: 0.9984 - auc: 0.9997 - precision: 0.8386 - recall: 1.0000

[0.004388206638395786,

0.9983686804771423,

0.9997254014015198,

0.8385772705078125,

1.0]

==========

model.evaluate(test_generator)

40/40 [==========] - 6s 154ms/step - loss: 0.3717 - accuracy: 0.9365 - auc: 0.9764 - precision: 0.6703 - recall: 0.9746

[0.3716888725757599,

0.9365079402923584,

0.976423978805542,

0.6703056693077087,

0.9746031761169434]

==========

```
model.evaluate(validation_generator)
```

12/12 [==============================] - 2s 135ms/step - loss: 2.3213 - accuracy: 0.7363 - auc: 0.8558 - precision: 0.5414 - recall: 0.7912

[2.3212876319885254,

0.7362637519836426,

0.855814516544342,

0.5413534045219421,

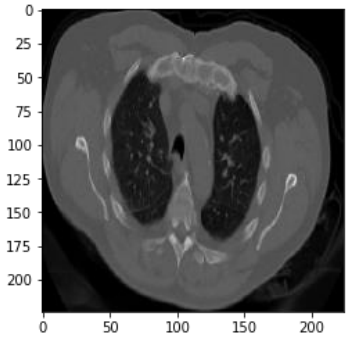
0.791208803653717]

```
==================================================================
model.save('/content/drive/MyDrive/ColabNotebooks/FinalYearProject/DenseNet169.h5',
model)
==================================================================
from keras.preprocessing import image
img=tf.keras.utils.load_img('/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid/squamous.cell.carcinoma_left.hilum_T1_N2_M0_IIIa/000117 (4).png',target_size=(224,224))
imag = tf.keras.utils.img_to_array(img)
imaga = np.expand_dims(imag,axis=0)
ypred = model.predict(imaga)
print(ypred)
a=np.argmax(ypred,-1)
if a==0:
  op="Adenocarcinoma"
elif a==1:
  op="large cell carcinoma"
elif a==2:
  op="normal (void of cancer)"
else:
  op="squamous cell carcinoma"
plt.imshow(img)
print("THE UPLOADED IMAGE IS SUSPECTED AS: "+str(op))
```

1/1 [==============================] - 0s 33ms/step

[[0.14293109 0.00412542 0.03430203 0.9988348]]

THE UPLOADED IMAGE IS SUSPECTED AS: squamous cell carcinoma



5.3 MobileNetV2

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

==========

```
import numpy as np
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from keras import Sequential
from tensorflow.keras.layers import *
from tensorflow.keras.models import *
from tensorflow.keras.preprocessing import image
```

==========

```
train_datagen = image.ImageDataGenerator(
    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
```

```
test_datagen= image.ImageDataGenerator(    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
================================================================
train_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/train',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
test_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/test',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
validation_generator = test_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid',
    target_size = (224,224),
    batch_size = 8,
    shuffle=True,
    class_mode = 'categorical')
Found 613 images belonging to 4 classes.
Found 315 images belonging to 4 classes.
Found 91 images belonging to 4 classes.
================================================================
base_model=tf.keras.applications.MobileNetV2(weights='imagenet',input_shape=(224,224,3)
, include_top=False)
for layer in base_model.layers:
    layer.trainable=True
```

```
model = Sequential()
model.add(base_model)
model.add(GaussianNoise(0.25))
model.add(GlobalAveragePooling2D())
model.add(Dense(1024,activation='relu'))
model.add(BatchNormalization())
model.add(GaussianNoise(0.25))
model.add(Dropout(0.25))
model.add(Dense(4, activation='sigmoid'))
model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5

9406464/9406464 [==============================] - 0s 0us/step

Model: "sequential"

Layer (type)	*Output Shape*	*Param #*
mobilenetv2_1.00_224 (Functional)	*(None, 7, 7, 1280)*	*2257984*
gaussian_noise (GaussianNoise)	*(None, 7, 7, 1280)*	*0*
global_average_pooling2d (GlobalAveragePooling2D)	*(None, 1280)*	*0*
dense (Dense)	*(None, 1024)*	*1311744*
batch_normalization (BatchNormalization)	*(None, 1024)*	*4096*
gaussian_noise_1 (GaussianNoise)	*(None, 1024)*	*0*
dropout (Dropout)	*(None, 1024)*	*0*
dense_1 (Dense)	*(None, 4)*	*4100*

Total params: 3,577,924

Trainable params: 3,541,764

Non-trainable params: 36,160

==========

```
model.compile(loss='categorical_crossentropy',
              optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
              metrics=['accuracy','AUC','Precision','Recall'])
```

==========

```
from keras.callbacks import EarlyStopping
es=EarlyStopping(monitor='val_loss',patience=3)
history = model.fit(
    train_generator,
    epochs=100,
    validation_data=validation_generator,
    steps_per_epoch= 75   )
```

Epoch 1/100

75/75 [==========] - 175s 2s/step - loss: 1.8308 - accuracy: 0.3199 - auc: 0.5742 - precision: 0.2899 - recall: 0.5846 - val_loss: 1.4680 - val_accuracy: 0.2637 - val_auc: 0.5580 - val_precision: 0.2605 - val_recall: 0.3407

Epoch 2/100

75/75 [==========] - 14s 187ms/step - loss: 1.4816 - accuracy: 0.4372 - auc: 0.6674 - precision: 0.3409 - recall: 0.6750 - val_loss: 1.5669 - val_accuracy: 0.3516 - val_auc: 0.5693 - val_precision: 0.3178 - val_recall: 0.4505....

....

…

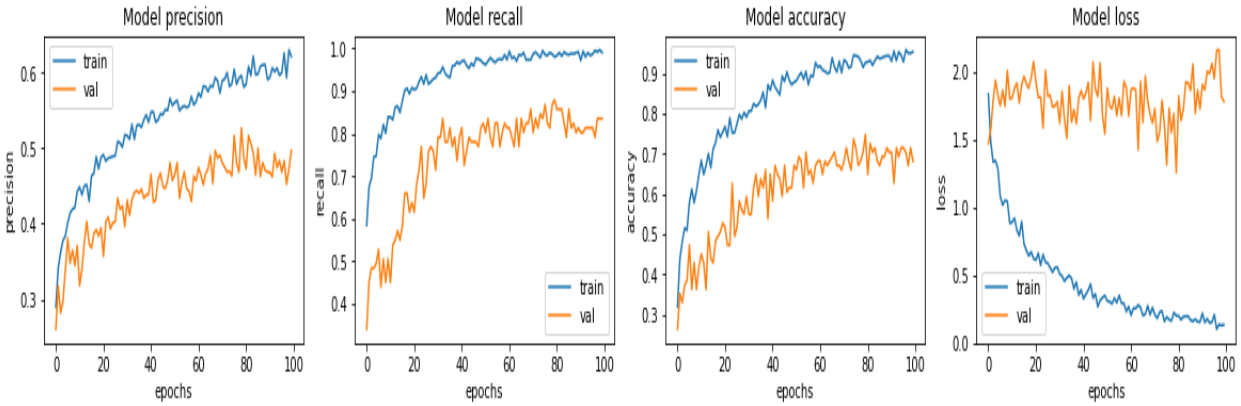
Epoch 100/100

75/75 [==========] - 12s 159ms/step - loss: 0.1320 - accuracy: 0.9531 - auc: 0.9899 - precision: 0.6215 - recall: 0.9899 - val_loss: 1.7782 - val_accuracy: 0.6813 - val_auc: 0.8652 - val_precision: 0.4967 - val_recall: 0.8352

==========

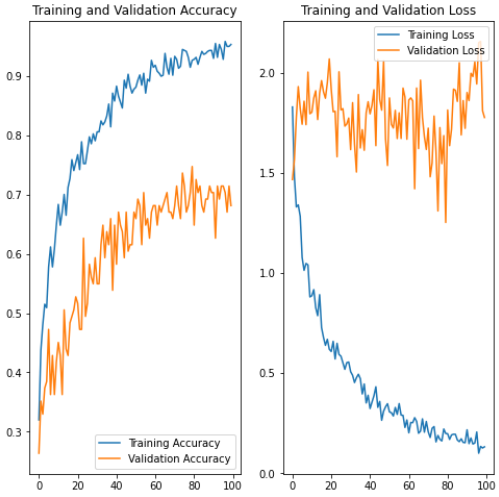
```
fig,ax=plt.subplots(1,4,figsize=(20,3))
ax=ax.ravel()
for i,met in enumerate(['precision','recall','accuracy','loss']):
  ax[i].plot(history.history[met])
```

```
ax[i].plot(history.history['val_'+met])
ax[i].set_title('Model {}'.format(met))
ax[i].set_xlabel('epochs')
ax[i].set_ylabel(met)
ax[i].legend(['train', 'val'])
```



==========

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(100)
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```



==========

model.evaluate(train_generator)

77/77 [==========] - 10s 125ms/step - loss: 0.0162 - accuracy: 0.9951 - auc: 0.9995 - precision: 0.7503 - recall: 1.0000

[0.01617632806301117,

0.995106041431427,

0.9995312094688416,

0.7503060102462769,

1.0]

==========

model.evaluate(test_generator)

40/40 [==========] - 112s 3s/step - loss: 0.5010 - accuracy: 0.8635 - auc: 0.9549 - precision: 0.5267 - recall: 0.9714

[0.5010098218917847,

0.8634920716285706,

0.9549391269683838,

0.5266781449317932,

0.9714285731315613]

==========

model.evaluate(validation_generator)

12/12 [==========] - 1s 119ms/step - loss: 2.1412 - accuracy: 0.6813 - auc: 0.8320 - precision: 0.4706 - recall: 0.7912

[2.1411569118499756,

0.6813187003135681,

0.8319848775863647,

0.47058823704719543,

0.791208803653717]

==========

model.save('/content/drive/MyDrive/ColabNotebooks/FinalYearProject/MobileNetV2.h5',

model)

==========

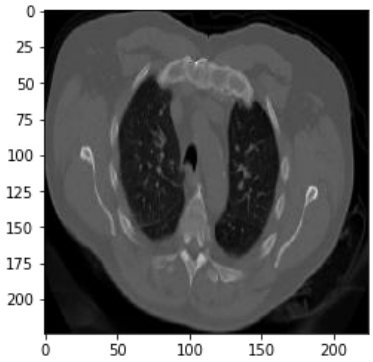
```
from keras.preprocessing import image
img=tf.keras.utils.load_img('/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid/squamous.cell.carcinoma_left.hilum_T1_N2_M0_IIIa/000117 (4).png',target_size=(224,224))
imag = tf.keras.utils.img_to_array(img)
imaga = np.expand_dims(imag,axis=0)
ypred = model.predict(imaga)
print(ypred)
a=np.argmax(ypred,-1)
if a==0:
  op="Adenocarcinoma"
elif a==1:
  op="large cell carcinoma"
elif a==2:
  op="normal (void of cancer)"
else:
  op="squamous cell carcinoma"
plt.imshow(img)

print("THE UPLOADED IMAGE IS SUSPECTED AS: "+str(op))
```

1/1 [==============================] - 3s 3s/step

[[0.8957554 0.07617089 0.13544597 0.8682142]]

THE UPLOADED IMAGE IS SUSPECTED AS: squamous cell carcinoma



5.4 ResNet152

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
==================================================================
import numpy as np
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from keras import Sequential
from tensorflow.keras.layers import *
from tensorflow.keras.models import *
from tensorflow.keras.preprocessing import image
==================================================================
train_datagen = image.ImageDataGenerator(
    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
test_datagen= image.ImageDataGenerator(    rotation_range=15,
```

```
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
================================================================
train_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/train',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
test_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/test',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
validation_generator = test_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid',
    target_size = (224,224),
    batch_size = 8,
    shuffle=True,
    class_mode = 'categorical')
Found 613 images belonging to 4 classes.
Found 315 images belonging to 4 classes.
Found 91 images belonging to 4 classes.
================================================================
base_model=tf.keras.applications.ResNet152(weights='imagenet',input_shape=(224,224,3),
include_top=False)
for layer in base_model.layers:
    layer.trainable=True
model = Sequential()
```

```
model.add(base_model)
model.add(GaussianNoise(0.25))
model.add(GlobalAveragePooling2D())
model.add(Dense(1024,activation='relu'))
model.add(BatchNormalization())
model.add(GaussianNoise(0.25))
model.add(Dropout(0.25))
model.add(Dense(4, activation='sigmoid'))
model.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet152_weights_tf_dim_ordering_tf_kernels_notop.h5
234698864/234698864 [==============================] - 12s 0us/step
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 resnet152 (Functional)      (None, 7, 7, 2048)        58370944
 gaussian_noise (GaussianNoise)  (None, 7, 7, 2048)    0
 global_average_pooling2d (G  (None, 2048)             0
 lobalAveragePooling2D)
 dense (Dense)               (None, 1024)              2098176
 batch_normalization (BatchN  (None, 1024)             4096
 ormalization)
 gaussian_noise_1 (GaussianN  (None, 1024)             0
 oise)
 dropout (Dropout)           (None, 1024)              0
 dense_1 (Dense)             (None, 4)                 4100
=================================================================
Total params: 60,477,316
Trainable params: 60,323,844
Non-trainable params: 153,472
_________________________________________________________________
```

```
        model.compile(loss='categorical_crossentropy',
          optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
          metrics=['accuracy','AUC','Precision','Recall'])
```

==========

```
from keras.callbacks import EarlyStopping
es=EarlyStopping(monitor='val_loss',patience=3)
history = model.fit(
    train_generator,
    epochs=100,
    validation_data=validation_generator,
    steps_per_epoch= 75   )
```

Epoch 1/100

75/75 [==========] - 316s 4s/step - loss: 1.5608 - accuracy: 0.4121 - auc: 0.6464 - precision: 0.3291 - recall: 0.6449 - val_loss: 1.4529 - val_accuracy: 0.3516 - val_auc: 0.6211 - val_precision: 0.3228 - val_recall: 0.5604

Epoch 2/100

75/75 [==========] - 24s 320ms/step - loss: 1.1165 - accuracy: 0.5846 - auc: 0.7828 - precision: 0.4112 - recall: 0.7638 - val_loss: 1.4339 - val_accuracy: 0.3736 - val_auc: 0.6318 - val_precision: 0.2951 - val_recall: 0.5934

....

...

...

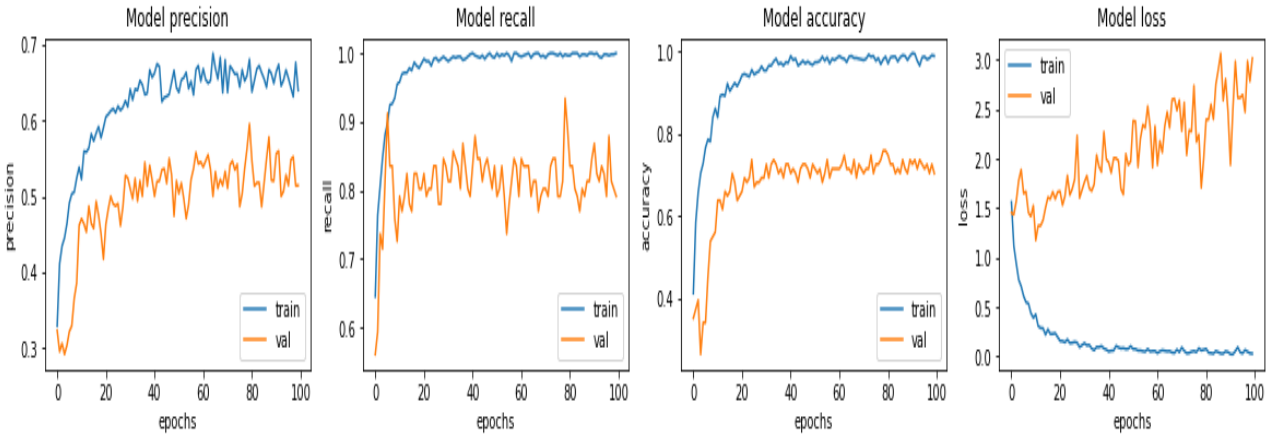
Epoch 100/100

75/75 [==========] - 21s 275ms/step - loss: 0.0290 - accuracy: 0.9883 - auc: 0.9975 - precision: 0.6399 - recall: 1.0000 - val_loss: 3.0141 - val_accuracy: 0.7033 - val_auc: 0.8422 - val_precision: 0.5143 - val_recall: 0.7912

==========

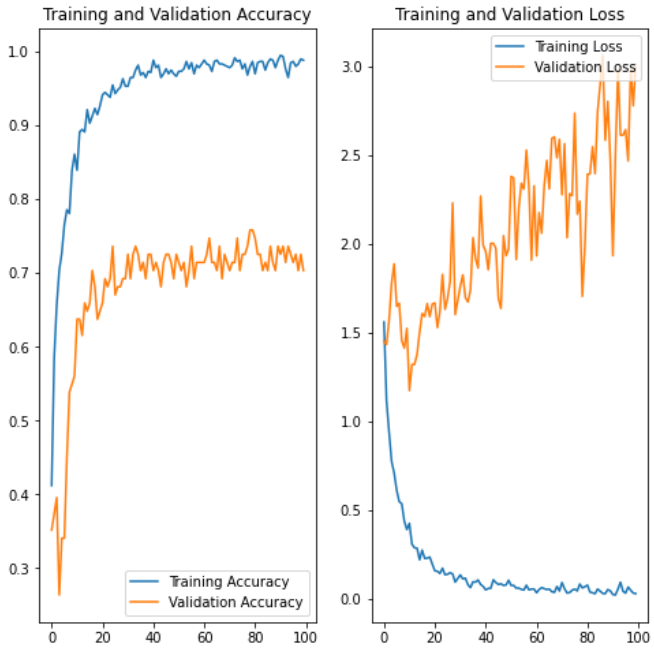
```
fig,ax=plt.subplots(1,4,figsize=(20,3))
ax=ax.ravel()
for i,met in enumerate(['precision','recall','accuracy','loss']):
  ax[i].plot(history.history[met])
  ax[i].plot(history.history['val_'+met])
  ax[i].set_title('Model {}'.format(met))
  ax[i].set_xlabel('epochs')
```

```
ax[i].set_ylabel(met)
ax[i].legend(['train', 'val'])
```



==========

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(100)
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```



==========

model.evaluate(train_generator)

77/77 [==========] - 11s 148ms/step - loss: 0.0061 - accuracy: 0.9984 - auc: 0.9986 - precision: 0.7711 - recall: 1.0000

[0.0060591851361095905,

0.9983686804771423,

0.9985730648040771,

0.7710691690444946,

1.0]

==========

model.evaluate(test_generator)

40/40 [==========] - 6s 149ms/step - loss: 0.2741 - accuracy: 0.9619 - auc: 0.9846 - precision: 0.6865 - recall: 0.9873

[0.27414458990097046,

0.961904764175415,

0.9845855832099915,

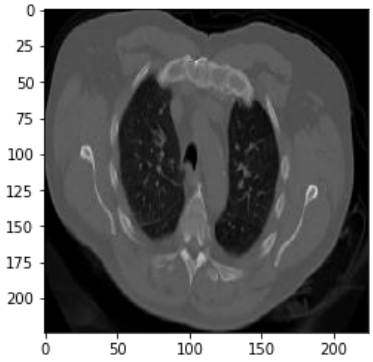
0.6865342259407043,

0.9873015880584717]

==========

```
model.evaluate(validation_generator)
12/12 [==============================] - 2s 134ms/step - loss: 3.0188 - accuracy: 0.7253 - auc: 0.8329 - precision: 0.4863 - recall: 0.7802
[3.0187904834747314,
 0.7252747416496277,
 0.8328704237937927,
 0.48630136251449585,
 0.7802197933197021]
==========================================================================
model.save('/content/drive/MyDrive/ColabNotebooks/FinalYearProject/ResNet152.h5',
model)
==========================================================================
from keras.preprocessing import image
img=tf.keras.utils.load_img('/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid/squamous.cell.carcinoma_left.hilum_T1_N2_M0_IIIa/000117 (4).png',target_size=(224,224))
imag = tf.keras.utils.img_to_array(img)
imaga = np.expand_dims(imag,axis=0)
ypred = model.predict(imaga)
print(ypred)
a=np.argmax(ypred,-1)
if a==0:
  op="Adenocarcinoma"
elif a==1:
  op="large cell carcinoma"
elif a==2:
  op="normal (void of cancer)"
else:
  op="squamous cell carcinoma"
plt.imshow(img)
print("THE UPLOADED IMAGE IS SUSPECTED AS: "+str(op))
1/1 [==============================] - 3s 3s/step
[[1.6535861e-04 2.1244949e-03 5.6799300e-02 9.9994051e-01]]
```

THE UPLOADED IMAGE IS SUSPECTED AS: squamous cell carcinoma



5.5 VGG19

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

==========

```
import numpy as np
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from keras import Sequential
from tensorflow.keras.layers import *
from tensorflow.keras.models import *
from tensorflow.keras.preprocessing import image
```

==========

```
train_datagen = image.ImageDataGenerator(
    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
```

```
test_datagen= image.ImageDataGenerator(    rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
================================================================
train_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/train',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
test_generator = train_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/test',
    target_size = (224,224),
    batch_size = 8,
    class_mode = 'categorical')
validation_generator = test_datagen.flow_from_directory(
    '/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid',
    target_size = (224,224),
    batch_size = 8,
    shuffle=True,
    class_mode = 'categorical')
Found 613 images belonging to 4 classes.
Found 315 images belonging to 4 classes.
Found 91 images belonging to 4 classes.
================================================================
base_model=tf.keras.applications.VGG19(weights='imagenet',input_shape=(224,224,3),
include_top=False)
for layer in base_model.layers:
    layer.trainable=True
```

```
model = Sequential()
model.add(base_model)
model.add(GaussianNoise(0.25))
model.add(GlobalAveragePooling2D())
model.add(Dense(1024,activation='relu'))
model.add(BatchNormalization())
model.add(GaussianNoise(0.25))
model.add(Dropout(0.25))
model.add(Dense(4, activation='sigmoid'))
model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg19/vgg19_weights_tf_dim_ordering_tf_kernels_notop.h5

80134624/80134624 [==============================] - 4s 0us/step

Model: "sequential"

Layer (type)	*Output Shape*	*Param #*
vgg19 (Functional)	*(None, 7, 7, 512)*	*20024384*
gaussian_noise (GaussianNoise)	*(None, 7, 7, 512)*	*0*
global_average_pooling2d (GlobalAveragePooling2D)	*(None, 512)*	*0*
dense (Dense)	*(None, 1024)*	*525312*
batch_normalization (BatchNormalization)	*(None, 1024)*	*4096*
gaussian_noise_1 (GaussianNoise)	*(None, 1024)*	*0*
dropout (Dropout)	*(None, 1024)*	*0*
dense_1 (Dense)	*(None, 4)*	*4100*

Total params: 20,557,892

Trainable params: 20,555,844

Non-trainable params: 2,048

```
================================================================
model.compile(loss='categorical_crossentropy',
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
        metrics=['accuracy','AUC','Precision','Recall'])
================================================================
from keras.callbacks import EarlyStopping
es=EarlyStopping(monitor='val_loss',patience=3)
history = model.fit(
   train_generator,
   epochs=100,
   validation_data=validation_generator,
   steps_per_epoch= 75    )
```

Epoch 1/100

75/75 [==============================] - 347s 4s/step - loss: 1.6272 - accuracy: 0.3484 - auc: 0.5810 - precision: 0.2963 - recall: 0.5712 - val_loss: 1.6698 - val_accuracy: 0.4066 - val_auc: 0.6641 - val_precision: 0.2633 - val_recall: 0.9780

Epoch 2/100

75/75 [==============================] - 19s 259ms/step - loss: 1.3228 - accuracy: 0.4355 - auc: 0.6570 - precision: 0.3356 - recall: 0.6633 - val_loss: 2.3538 - val_accuracy: 0.3077 - val_auc: 0.4457 - val_precision: 0.2343 - val_recall: 0.9011

....

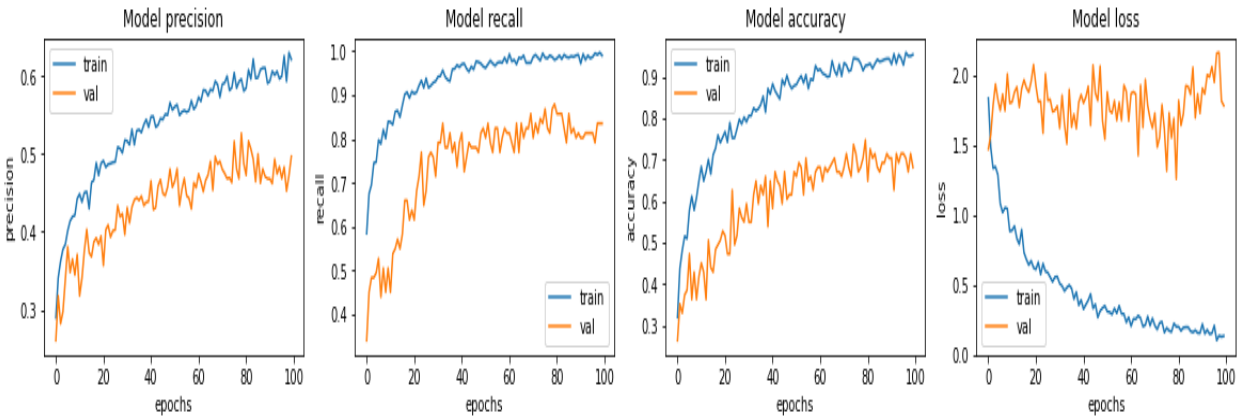
...

Epoch 100/100

75/75 [==============================] - 15s 201ms/step - loss: 0.7457 - accuracy: 0.6817 - auc: 0.8705 - precision: 0.4531 - recall: 0.8978 - val_loss: 1.2990 - val_accuracy: 0.4945 - val_auc: 0.7435 - val_precision: 0.3550 - val_recall: 0.7802

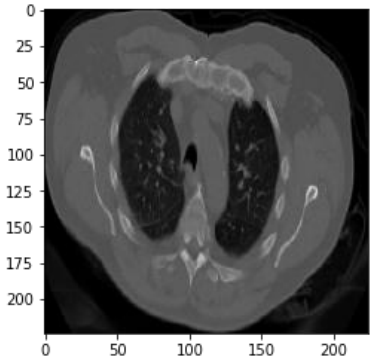
```
================================================================
fig,ax=plt.subplots(1,4,figsize=(20,3))
ax=ax.ravel()
for i,met in enumerate(['precision','recall','accuracy','loss']):
  ax[i].plot(history.history[met])
  ax[i].plot(history.history['val_'+met])
  ax[i].set_title('Model {}'.format(met))
  ax[i].set_xlabel('epochs')
```

```
ax[i].set_ylabel(met)
ax[i].legend(['train', 'val'])
```



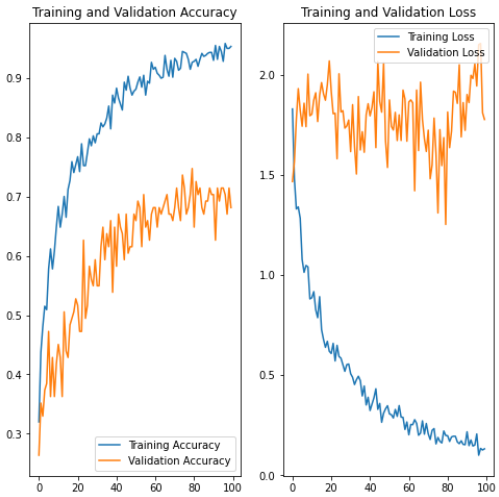
==========

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(100)
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
```



plt.title('Training and Validation Loss')

plt.show()



==========

model.evaluate(train_generator)

77/77 [==========] - 11s 136ms/step - loss: 0.8026 - accuracy: 0.6688 - auc: 0.8699 - precision: 0.4241 - recall: 0.8940

[0.8025810718536377,

0.6688417792320251,

0.8698943257331848,

0.4241486191749573,

0.8939641118049622]

==========

model.evaluate(test_generator)

40/40 [==========] - 225s 6s/step - loss: 1.2117 - accuracy: 0.4952 - auc: 0.7650 - precision: 0.3679 - recall: 0.7651

[1.2116541862487793,

0.4952380955219269,

0.7650474309921265,

0.36793893575668335,

0.7650793790817261]

==========

model.evaluate(validation_generator)

12/12 [==============================] - 2s 180ms/step - loss: 1.3617 - accuracy: 0.3956 - auc: 0.7350 - precision: 0.3622 - recall: 0.7802

[1.361678123474121,

0.3956044018268585,

0.735015869140625,

0.3622449040412903,

0.7802197933197021]

==========

```
model.save('/content/drive/MyDrive/ColabNotebooks/FinalYearProject/VGG19.h5',
model)
```

==========

```
from keras.preprocessing import image
img=tf.keras.utils.load_img('/content/drive/MyDrive/Colab Notebooks/Final Year Project/Dataset_1/valid/squamous.cell.carcinoma_left.hilum_T1_N2_M0_IIIa/000117 (4).png',target_size=(224,224))
imag = tf.keras.utils.img_to_array(img)
imaga = np.expand_dims(imag,axis=0)
ypred = model.predict(imaga)
print(ypred)
a=np.argmax(ypred,-1)
if a==0:
  op="Adenocarcinoma"
elif a==1:
  op="large cell carcinoma"
elif a==2:
  op="normal (void of cancer)"
else:
  op="squamous cell carcinoma"
plt.imshow(img)
print("THE UPLOADED IMAGE IS SUSPECTED AS: "+str(op))
```

1/1 [==============================] - 1s 897ms/step

[[0.5365608 0.51767033 0.0043244 0.92249805]]

THE UPLOADED IMAGE IS SUSPECTED AS: squamous cell carcinoma

CHAPTER 6

TESTING

6.1 Software Testing

The following software testing techniques were used-

6.1.1 Test Adequacy Criteria for DL-

To address the challenge mentioned above, test adequacy criteria have been proposed for DL. Inspired by traditional code coverage metrics such as statement or decision coverage, neuron coverage counts the number of activated neurons in the neural network when test inputs are submitted for classification. The study reports that the higher the neuron activation coverage, the higher the chances of detecting wrong classifications. Other studies have confirmed this initial result and also extended the proposed structural criteria by distinguishing neuron-level criteria from layer-level coverage criteria. Instead of looking at the network structure, a different criteria called surprise adequacy has been introduced for measuring the diversity in training data. The surprise of an input is defined as the difference between the input and the training dataset w.r.t. the behavior of the DL model. It is then suggested to select inputs which are sufficiently surprising but still within the expected data distribution to compose an adequate test set. These criteria are specific to DL and can be used to guide the selection of test inputs, but techniques are needed to evaluate the correctness of results when test inputs are submitted to DL models, which we will discuss in the following.

Code-

```
model.add(Dense(1024,activation='relu'))
```

Where 1024 denotes number of neurons, Higher the neurons activation coverage higher the chances of detecting wrong classification

6.1.2 Differential Testing-

In Differential Testing (DT) the functionality of a system is assessed by comparing the behaviour of multiple implementations or models for the same task. All these systems are executed with the same inputs and the difference between their results is used to assess the correctness of the tested program.DT’s main issues are 1) how to identify the faulty system, as there is no way to determine which system provides the correct answer and 2) how to exploit a test set of sufficient quality to get confidence in the correctness of the tested system.

On applying differential testing ResNet Model Performed the best.

6.1.3 Metamorphic Testing

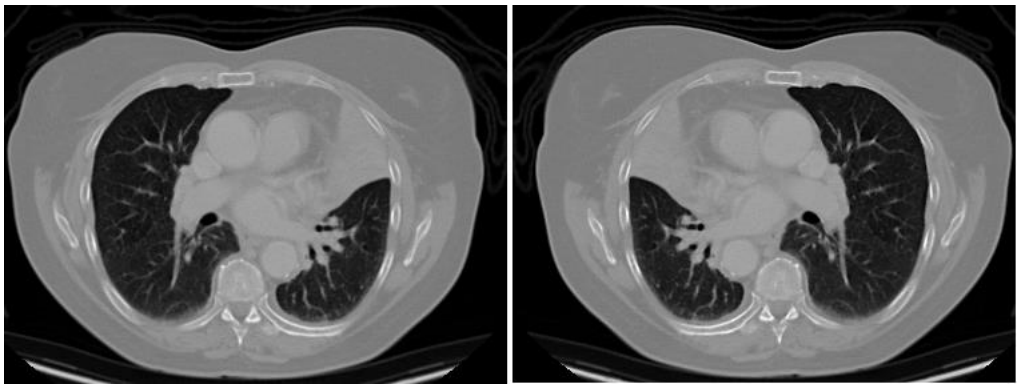


Fig 6.1 Metamorphic Testing

Metamorphic Testing (MT) is an effective method to deal with programs for which an oracle is difficult or even impossible to define. In MT, known relations between the inputs and outputs of a program, called metamorphic relations (MR), can be used to find faults in the program [17]. For example, in Figure 2, a typical MR says that applying horizontal flipping over the image should not change its classification label, even though the correct label is unknown. Starting from an initial source test case, by using MRs, it is possible to generate follow-up test cases for which the expected result has to satisfy properties defined by the MRs. If the relation is violated, then a fault is revealed, even though we ignore where is the fault located and what is the correct expected output of the program. While an exact test oracle is usually impossible to get for DL, due to the stochastic nature of the predictions computed by the model, many MRs can often easily be identified and conjointly used to check the model outputs.

6.1.4 Mutation Testing (MuT)-

Mutation Testing (MuT) is a traditional testing method, used to assess the quality of a test set and to generate fault-revealing test cases. In MuT, mutants are modified versions of the program under test and the goal is to evaluate a test suite by its ability to distinguish the original program from its mutants. A mutant is generated by applying one or multiple mutation operators on the original program, i.e., by only introducing minor changes in the data set or the model architecture [25]. When a test case observes a difference in the behaviour of the original program and its mutant, we say that it ”kills” the mutant.

Eg- train_datagen = image.ImageDataGenerator(

rotation_range=15,

shear_range=0.2,

zoom_range=0.2,

```
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
test_datagen= image.ImageDataGenerator(rotation_range=15,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    width_shift_range=0.1,
    height_shift_range=0.1)
```

6.1.4 Combinatorial Testing-

Combinatorial Testing (CT) is a software testing approach that uses value interactions coverage of input data to measure the quality of test suites and evaluate software system. CT follows the assumption that input parameters often interact with each other and certain tuples of values may actually lead to faults. Focusing on dedicated interactions between parameters (typically pairs or triple of values) on the basis of representative values instead of performing an exhaustive search of the whole input space allows thereby a more structured but also more efficient testing of the system. This efficiency advantage has also been a motivating factor for the application of CT in DL systems. Here, CT is considered to be a black-box testing technique which delves into the network to co-relate between architectural neurons. CT is of two types; fixed strength and variable strength CT both of which have been explored.

6.1.5 Adversarial Perturbation Testing (APT)-

A technique developed to attack machine learning systems, known as adversarial perturbation can be tuned to serve the purpose of testing DL models. An adversarial example is an intentional attack created to fool a DL model with a test input which makes the system produce a wrong answer. Interestingly, adversarial inputs are close to correctly classified inputs, so that they remain undetectable for human eyes or classical detection means.

CHAPTER 7

RESULT ANALYSIS

Here are the models along with their accuracy used in system:

1. VGG- The VGG19 model along with the current dataset results in an accuracy of 49.52% , Precision of 36.79%, Loss of 121.17% and Recall of 76.51%.

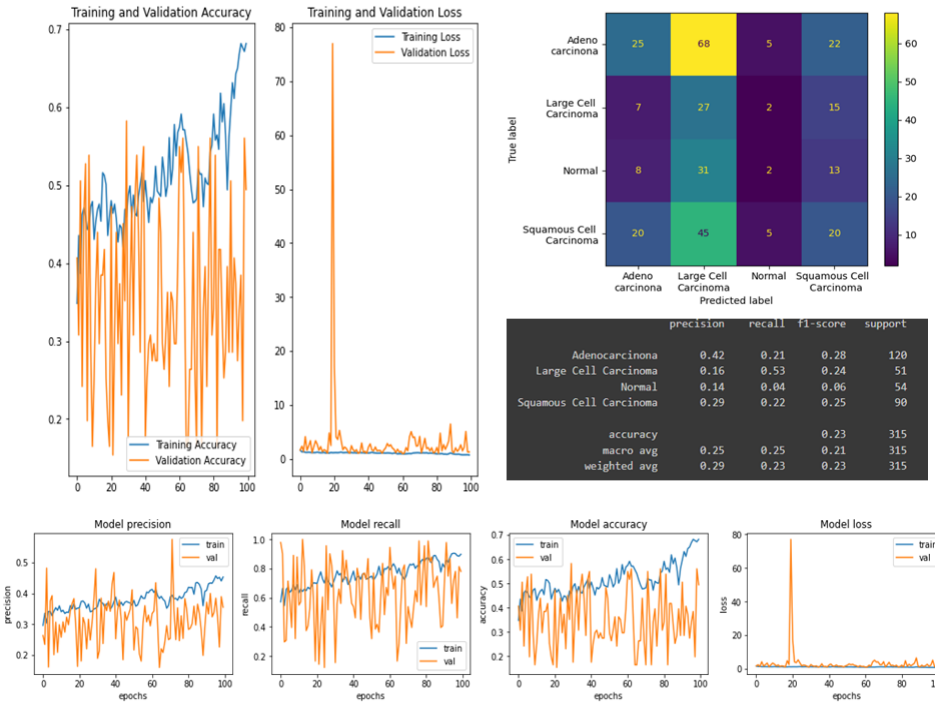


Fig 7.1 VGG Results

2. ResNet- The ResNet152 model along with the current dataset results in an accuracy of 96.19% , Precision of 58.21%, Loss of 24.72% and Recall of 99.05%.

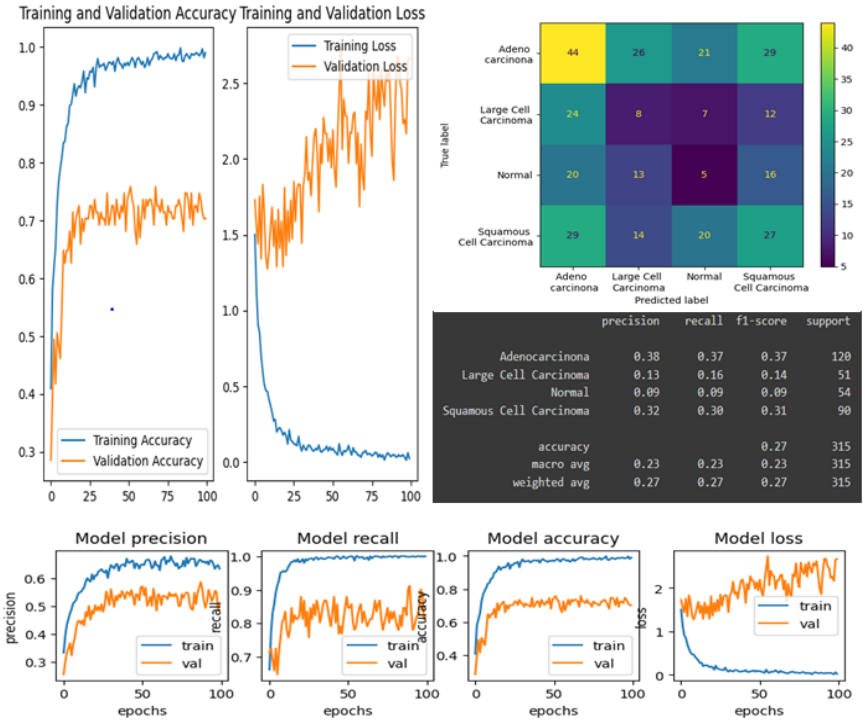


Fig 7.2 ResNet Results

3. The MobileNetV2 model along with the current dataset results in an accuracy of 86.35% , Precision of 52,67%, Loss of 50.10% and Recall of 97.14%.

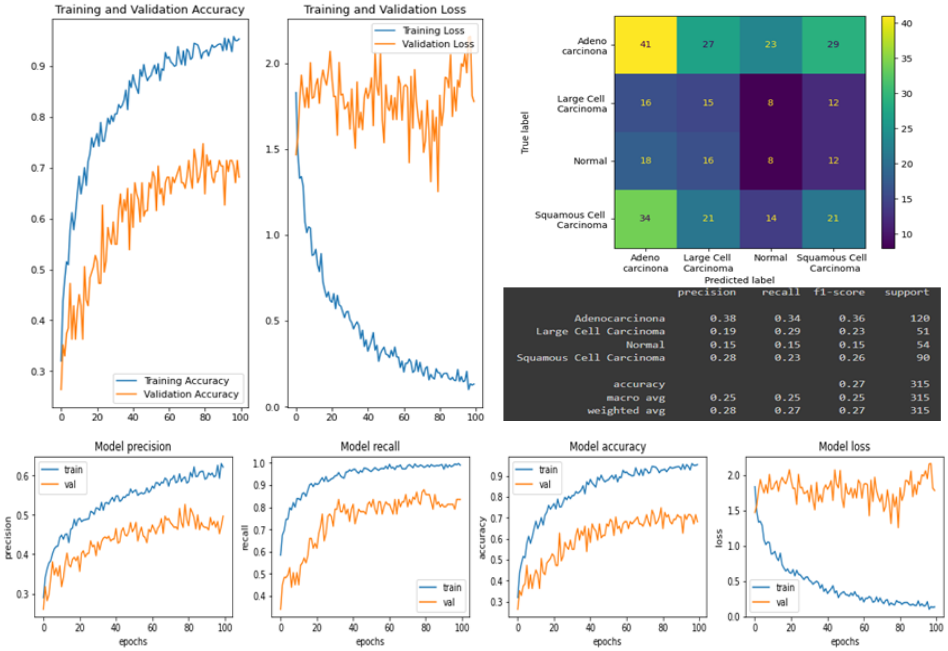


Fig 7.3 MobileNet Results

4. EfficientNet- The EfficientNetB3 model along with the current dataset results in an accuracy of 90.16% , Precision of 57.57%, Loss of 38.42% and Recall of 97.78 %

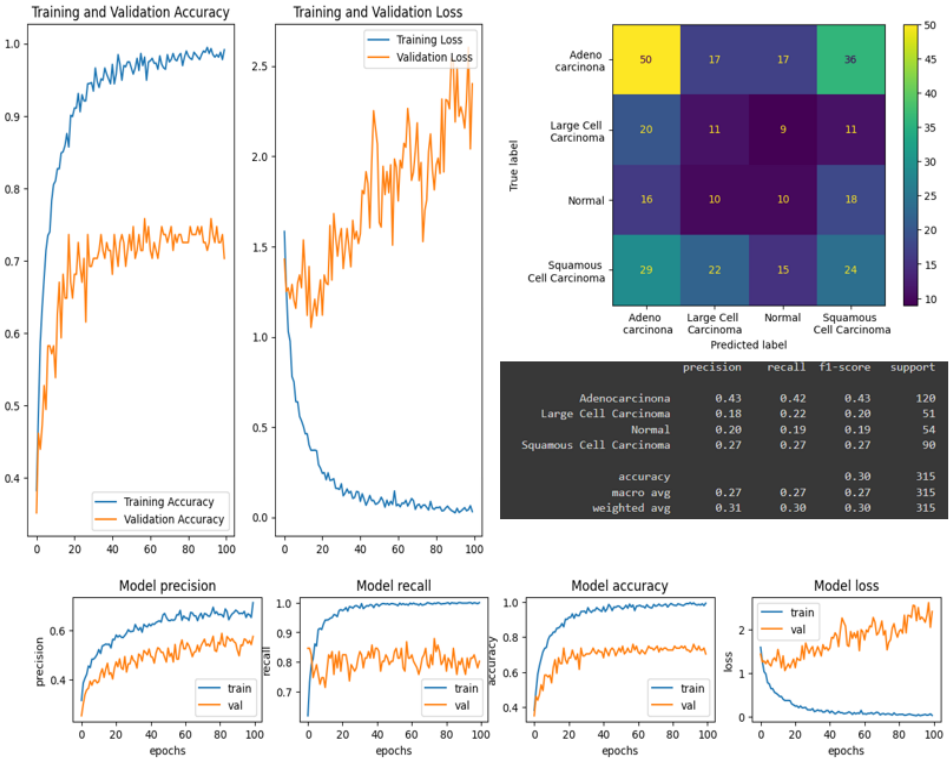


Fig 7.4 EfficientNet Results

5. DenseNet- The DenseNet169 model along with the current dataset results in an accuracy of 93.65% , Precision of 60/59%, Loss of 30.11% and Recall of 98.10%.

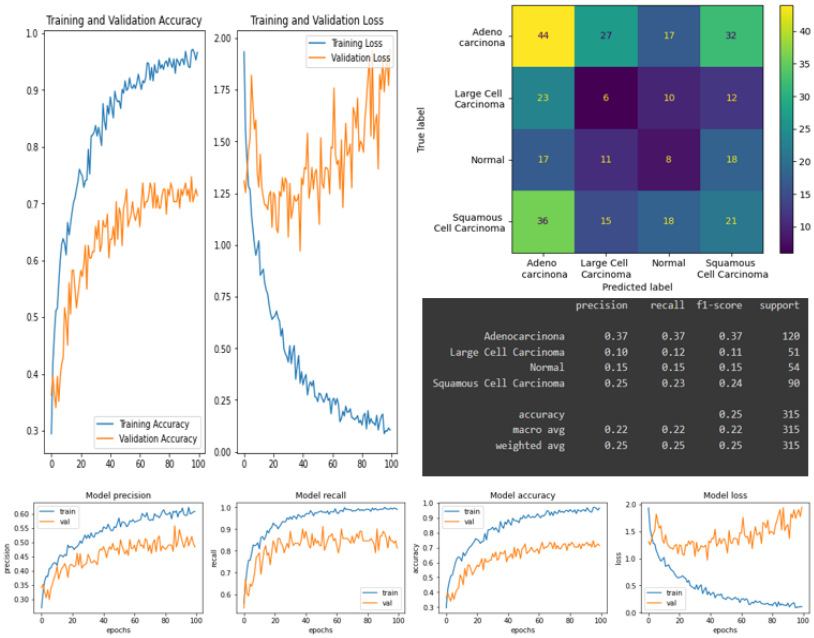


Fig 7.5 DenseNet Results

Further the used dataset is equalized for better enhancement of the images in the dataset and highlight the underlying features of the images by increasing the contrast.
Since the ResNet obtained the highest accuracy in the above process we will use ResNet for further processing.
ResNet Model us used with Equalized dataset and removing Gaussian for further analysis- The Model with above mentioned characteristics Obtains an accuracy of 90.07%, Precision of 53.51%, Loss of 53.25% and Recall of 96.03%.

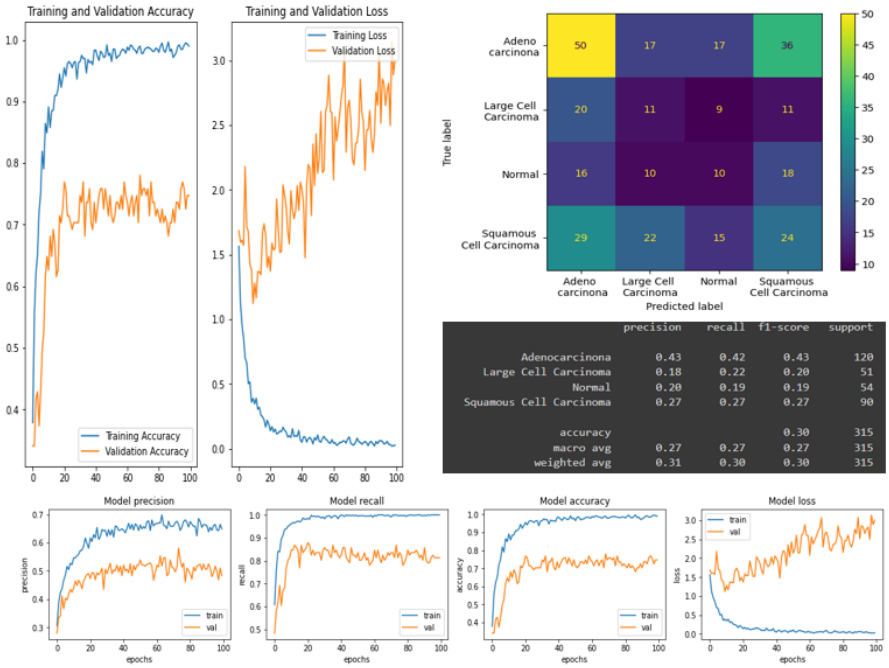


Fig 7.6 ResNet Results with Equalized dataset and Removing Gaussian

Since the accuracy decreased by removing gaussian and using and using equalized dataset we next tried to change the optimizer from Adam to SGD and ADAgrad.

The Model with SGD as optimizer obtained an accuracy of 54.60%, Precision of 43.41%, Loss of 128.72% and Recall of 71.11%

The Model ADAgrad as optimizer obtained an accuracy of 53.97%, Precision of 41.00%, Loss of 114.45% and Recall of 83.17%

CHAPTER 8

CONCLUSION

7.1 Result

An artificial Neural Network for diagnose the presence or absence of lung cancer in human body movie was developed. The models were tested and testing results are shown below in the table:

ResNet152	96.19
DenseNet169	93.65
VGG19	49.52
EfficientNetB3	90.16
MobileNetV2	86.35

Fig 8.1 Models before Equalization

Next the dataset was Equalized and equalized dataset was used on two highest testing accuracy models- ResNet152 and DenseNet169. The Results are shown below-

ResNet152	87.09
DenseNet169	91.39

Fig 8.2 Models after Equalization

Next the Gaussian was removed from the base model and Equalized dataset was used on ResNet152 and DenseNet169. The Results are shown below-

ResNet152	90.07
DenseNet169	92.05

Fig 8.3 Models after removing Gaussian

The best model with highest accuracy is ResNet152 which is 96.19% accurate. This shows that the neural network is able to diagnose lung cancer, so it can used as a diagnose tool by doctors.

7.2 Future Scope

This project goal is to detect and classify malignant and benign cancer cells using CT scan lung images. To detect the location of cancer cells, this work uses the capsule-based saliency segmentation and transfer learning-based feature extraction. Furthermore, the proposed architecture employs the whale-based classification layers to achieve better accuracy. The experimental results show that the proposed architecture has achieved the best results associated with other standard architectures and obtained the best peak results. In the future,

more vigorous testing is required using larger real-time clinical datasets. Additionally, the proposed algorithm needs improvisation in terms of computational complexity which will play a significant role in the analysis and identification of tumor cells as per radiologists' perspective more accurately in future.

CHAPTER-9

REFRENCES

REFRENCES

1. Non-Small Cell Lung Cancer, Available at: http://www.katemacintyrefoundation.org/pdf/non-small-cell.pdf, Adapted from National Cancer Institute (NCI) and Patients Living with Cancer (PLWC), 2007, (accessed July 2011)
2. Nasser, I. M., Al-Shawwa, M., & Abu-Naser, S. S. (2019). A Proposed Artificial Neural Network for Predicting Movies Rates Category. International Journal of Academic Engineering Research (IJAER), 3(2).
3. Nasser, I. M., & Abu-Naser, S. S. (2019). Predicting Tumor Category Using Artificial Neural Networks. International Journal of Academic Health and Medical Research (IJAHMR), 3(2).
4. L. Dormehl, "Digital Trends," 5 1 2019. [Online]. Available: https://www.digitaltrends.com/cool-tech/what-is-an-artificialneural-network/. [Accessed 15 3 2019].
5. I. E. Livieris, K. Drakopoulou and P. Pintelas, "Predicting students' performance using artificial neural networks," in 8th PanHellenic Conference with International Participation Information and Communication Technologies in Education, Volos, Greece, 2012.
6. K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, in: Proceedings of the International Conference on Learning Representations (ICLR), 2015.
7. Olaf Ronneberger, Philipp Fischer, and Thomas Brox, "U- Net: Convolutional Networks for Biomedical Image Segmentation", arXiv, 2015
8. Zhou JT, Pan SJ, Tsang IW, Yan Y. Hybrid heterogeneous transfer learning through deep learning. In: AAAI. 2014. p. 2213–20.
9. K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778
10. Gao Huang, Zhuang Liu, Laurens van der Maaten, "Densely Connected Convolutional Networks", arXiv, 2018